scenario_A_reproduce_v3_numpy

June 8, 2026

reference

We are using numpy as the data type of everything here.

[19]:
```
import warnings

warnings.filterwarnings(
    "ignore",
    message=".*'where' used without 'out'.*",
    category=UserWarning,
    module="scipy.optimize._trustregion_constr"
)

warnings.filterwarnings(
    "ignore",
    message=".*delta_grad == 0.0.*",
    category=UserWarning,
    module="scipy.optimize._differentiable_functions"
)
```

```
time: 0 ns (started: 2026-06-07 21:48:41 -10:00)
```

[2]:
```
%load_ext autotime

#   0. Imports & path setup
import sys, os, copy, time, warnings
# sys.path.insert(0, "/home/niels/FELsim_clone/backend")
CURRENT_DIR = os.getcwd()
BACKEND_DIR = CURRENT_DIR if os.path.basename(CURRENT_DIR) == "backend_orig"␣
 ↪else os.path.abspath(os.path.join(CURRENT_DIR, ".."))
sys.path.insert(0, BACKEND_DIR)
print(BACKEND_DIR)
os.makedirs(os.path.join(CURRENT_DIR, "../results"), exist_ok=True)

import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("module://matplotlib_inline.backend_inline")
import matplotlib.pyplot as plt
```

```
import matplotlib.gridspec as gridspec
from matplotlib.lines import Line2D

from ebeam import beam as ebeam_class
from beamline import lattice
from excelElements import ExcelElements
from beamOptimizer import beamOptimizer
from evolutionPlotter import EvolutionPlotter
from felsimAdapter import FELsimAdapter

#   Shared constants
# EXCEL_PATH     = "/home/niels/FELsim_clone/beam_excel/Beamline_elements_3.
 ↪xlsx"
# EXCEL_PATH     = "/home/niels/FELsim_clone/beam_excel/Beamline_elements_3.
 ↪xlsx"
EXCEL_PATH  = os.path.abspath(os.path.join(CURRENT_DIR, "../../beam_excel/
 ↪Beamline_elements_3.xlsx"))


BEAM_ENERGY    = 40.0    # MeV as the energy for a single electron
N_PARTICLES    = 1000    # number of electrons in the beam for simulation
SEED           = 42      # random seed for reproducibility
CURRENT_BOUNDS = (0.01, 1.5)   # current bound in the experiment heuristically

#   Relativistic factors
relat = lattice(1, fringeType=None)
relat.setE(E=BEAM_ENERGY)       # set the energy of the relativistic factor
GAMMA_REL = relat.gamma         # Lorentz factor as the mass ratio of the␣
 ↪electron and the static electron
BETA_REL  = relat.beta          # standardized electron velocity
NORM      = GAMMA_REL * BETA_REL # electron momentom ratio between the current␣
 ↪electron and the static one

#   Undulator matching targets (MkV FEL, K=1.2,  _u=2.3 cm)
K        = 1.2      # Undulator Parameter
LAMBDA_U = 2.3e-2   # wavelength, unit: m
## twiss parameters
BETA_YM  = GAMMA_REL / (K * 2 * np.pi / LAMBDA_U)   # matched  _y at UND␣
 ↪entrance, beam size in y direction
BETA_XM  = 1.4       # m  - matched  _x at UND entrance, beam size at the x␣
 ↪direction
ALPHA_XM = 0.47      # - Courant-Snyder  _x at UND entrance, beam size converge␣
 ↪at the direction x
ALPHA_YM = 0.0       # - Courant-Snyder  _y at UND entrance, beam waist is␣
 ↪achieved at the y direction
```

```
print(f"  = {GAMMA_REL:.4f},    = {BETA_REL:.6f},    = {NORM:.4f}")
print(f"Matched  _ym = {BETA_YM:.4f} m   (K={K},  _u={LAMBDA_U*100:.1f} cm)")
print(f"Matched  _xm = {BETA_XM:.4f} m,   _xm = {ALPHA_XM:.4f},   _ym =
 ↪{ALPHA_YM:.4f}")
```

```
The autotime extension is already loaded. To reload it, use:
  %reload_ext autotime
c:\Users\yilu2\my_files\research\FELsim\backend_orig
  = 79.2794,    = 0.999920,    = 79.2731
Matched  _ym = 0.2418 m   (K=1.2,  _u=2.3 cm)
Matched  _xm = 1.4000 m,   _xm = 0.4700,   _ym = 0.0000
time: 18.4 s (started: 2026-06-07 21:24:50 -10:00)
```

0.1 1 — Beam Distribution

Physics-based 6D Gaussian matching the UH linac reference parameters.

[3]:
```
%load_ext autotime

#   Beam parameters
epsilon_n         = 8.0      # ·mm·mrad  normalised emittance, inner attribute
 ↪of electron gun
x_std             = 0.8      # mm, initial standard deviation at x direction
y_std             = 0.8      # mm, initial standard deviation at y direction
f_RF              = 2856e6   # Hz, acceleration frequency
bunch_spread_ps   = 2.0      # ps, beam length
energy_spread_pct = 0.5      # %, energy standard deviation inside the beam
h                 = 5e9      # 1/s  chirp, longitudinal chirp for beam
 ↪compression

epsilon     = epsilon_n / NORM # geometric emittance
x_prime_std = epsilon / x_std # alpha_x=0 at the initial point, so that sigma_x
 ↪* sigma_x'=epsilon
y_prime_std = epsilon / y_std # alpha_y=0 at the initial point, so that sigma_y
 ↪* sigma_y'=epsilon
tof_std     = bunch_spread_ps * 1e-9 * f_RF # Time of Flight Standard
 ↪Deviation, change from cycle to mili-cycle, measuring how much the electron
 ↪bunch occupies the cycle
energy_std  = energy_spread_pct * 10 # transform from percent to per mill

ebeam_obj = ebeam_class()
PARTICLES = ebeam_obj.gen_6d_gaussian(
    0,
    [x_std, x_prime_std, y_std, y_prime_std, tof_std, energy_std],
    N_PARTICLES,
)
# Apply longitudinal chirp
```

```
PARTICLES[:, 5] += h * (PARTICLES[:, 4] / f_RF)

print(f"  = {epsilon:.5f}  ·mm·mrad  (geometric)")
print(f" _x={x_std:.2f} mm,  _x'={x_prime_std:.5f} mrad")
print(f" _y={y_std:.2f} mm,  _y'={y_prime_std:.5f} mrad")
print(f"Particles: {PARTICLES.shape}")
print(f"PARTICLES class name: {PARTICLES.__class__.__name__}")
print(f"PARTICLES data type: {PARTICLES.dtype}")
```

```
The autotime extension is already loaded. To reload it, use:
  %reload_ext autotime
  = 0.10092  ·mm·mrad  (geometric)
 _x=0.80 mm,  _x'=0.12615 mrad
 _y=0.80 mm,  _y'=0.12615 mrad
Particles: (1000, 6)
PARTICLES class name: ndarray
PARTICLES data type: float64
time: 0 ns (started: 2026-06-07 21:25:09 -10:00)
```

0.2 2 — Beamline & Element Index Map

The `ExcelElements` parser produces **138 FELsim elements**. Key reference indices: | Location | FELsim idx | z (m) | Physical meaning | |—|—|—|—| | End of 1st doublet region | 8–9 | 0.73–1.0 | First targets | | IP (focus) | 59 | 7.11 | Minimum envelope | | UND entrance | 117 | 12.39 | Undulator matching |

[4]:
```
%load_ext autotime

#   Load beamline & map quad indices
beamline_full = ExcelElements(EXCEL_PATH).create_beamline()
print(f"FELsim elements: {len(beamline_full)}")

s = 0.0
quad_info = []
for i, e in enumerate(beamline_full):
    cls = e.__class__.__name__
    L   = float(getattr(e, 'length', 0.0) or 0.0)
    if cls in ('qpfLattice', 'qpdLattice'):
        quad_info.append((i, cls, round(s, 5), round(s+L, 5),
                          round(float(getattr(e, 'current', 0) or 0), 5)))
    s += L

quad_indices = [q[0] for q in quad_info]
df_quads = pd.DataFrame(quad_info,␣
 ↪columns=['idx','type','s_start','s_end','current_A'])
print(f"Quadrupoles: {len(quad_indices)}")
print(df_quads.to_string(index=False))
```

```
#   Gold-standard currents from Eremey's FELsim S1 solution
FELSIM_S1_CURRENTS = {
     1: 0.8218,  3: 1.0430,
    10: 3.8834,
    16: 2.2396, 18: 4.9532, 20: 3.4258,
    27: 4.6657,
    33: 2.6942, 35: 2.6523, 37: 0.2768, 39: 0.2768, 41: 2.6523, 43: 2.6942,
    50: 4.6739,
    56: 3.1219, 58: 3.3129,
    61: 5.1775, 63: 4.0434,
    70: 4.6818,
    76: 3.9336, 78: 4.0787, 80: 0.0139,
    87: 1.3624, 93: 0.9452, 95: 2.8851, 97: 2.1921,
}
print(f"\nFELSIM_S1_CURRENTS: {len(FELSIM_S1_CURRENTS)} quadrupoles loaded")
```

```
The autotime extension is already loaded. To reload it, use:
  %reload_ext autotime
FELsim elements: 138
Quadrupoles: 26
 idx       type  s_start    s_end  current_A
   1 qpfLattice  0.35878  0.44768    0.88572
   3 qpdLattice  0.64453  0.73342    1.05289
  10 qpfLattice  2.03304  2.12194    3.81660
  16 qpdLattice  2.73535  2.82425    2.57981
  18 qpfLattice  2.86743  2.95633    4.56570
  20 qpdLattice  2.99951  3.08841    2.57981
  27 qpfLattice  3.55448  3.64338    4.49146
  33 qpdLattice  4.15187  4.24077    1.59614
  35 qpfLattice  4.29157  4.38047    3.08092
  37 qpdLattice  4.43127  4.52017    1.59614
  39 qpdLattice  4.93513  5.02403    1.59614
  41 qpfLattice  5.07483  5.16373    3.08092
  43 qpdLattice  5.21453  5.30343    1.59614
  50 qpfLattice  5.83098  5.91988    4.49072
  56 qpdLattice  6.36946  6.45836    2.25575
  58 qpfLattice  6.55775  6.64665    1.67186
  61 qpfLattice  7.32562  7.41452    0.02487
  63 qpdLattice  7.46992  7.55882    0.00100
  70 qpfLattice  8.10749  8.19639    4.49146
  76 qpdLattice  8.60769  8.69659    1.59614
  78 qpfLattice  8.73278  8.82168    3.50000
  80 qpdLattice  8.85788  8.94678    1.59614
  87 qpfLattice  9.62889  9.71779    3.81660
  93 qpdLattice 10.21208 10.30098    1.59614
  95 qpfLattice 10.60921 10.69811    3.08092
  97 qpdLattice 10.73621 10.82511    1.59614
```

```
FELSIM_S1_CURRENTS: 26 quadrupoles loaded
time: 719 ms (started: 2026-06-07 21:25:09 -10:00)
```

[5]:
```
%load_ext autotime

#   Confirm key reference positions
s = 0.0
for i, e in enumerate(beamline_full):
    s += float(getattr(e, 'length', 0.0) or 0.0)
    if i in (8, 9, 15, 25, 32, 37, 55, 59, 92, 117):
        print(f"  el {i:3d}  {e.__class__.__name__:20s}  s_end={s:.5f} m")
```

```
The autotime extension is already loaded. To reload it, use:
  %reload_ext autotime
  el   8  dipole_wedge          s_end=1.80682 m
  el   9  driftLattice          s_end=2.03304 m
  el  15  driftLattice          s_end=2.73535 m
  el  25  dipole_wedge          s_end=3.34855 m
  el  32  driftLattice          s_end=4.15187 m
  el  37  qpdLattice            s_end=4.52017 m
  el  55  driftLattice          s_end=6.36946 m
  el  59  driftLattice          s_end=7.11013 m
  el  92  driftLattice          s_end=10.21208 m
  el 117  driftLattice          s_end=12.39158 m
time: 0 ns (started: 2026-06-07 21:25:10 -10:00)
```

0.3 3 — Statistical Benchmark Infrastructure

`run_benchmark()` runs the same scenario `N_RUNS` times from different random starting currents. `plot_stat_convergence()` plots **mean ± 1 RMS bands** — the same visual style as Eremey's reference convergence figures.

[6]:
```
%load_ext autotime

import time
import copy
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D

def method_label(method, label=None):
    if label:
        return label
    if isinstance(method, str):
        return method
    return getattr(method, "__name__", str(method))
```

```
#   Benchmark runner
def run_benchmark(scenario_name, beamline_slice_len, particles, seg_var, obj,␣
 ↪bounds,
                  method, n_runs=5, SEED=42, seed_offset=0, jac=None,␣
 ↪options=None, verbose=True, method_tag=None):
    """Run beamOptimizer.calc() n_runs times from uniform-random starting␣
 ↪currents."""
    var_names = list({seg_var[i][0] for i in seg_var})
    results   = []
    method_name = method_label(method, method_tag)

    for run_i in range(n_runs):
        bl = ExcelElements(EXCEL_PATH).create_beamline()[:beamline_slice_len]
        p  = particles.copy()
        rng = np.random.default_rng(SEED + seed_offset + run_i * 997)
        sp  = {v: {"bounds": bounds[v],
                   "start": float(rng.uniform(bounds[v][0], bounds[v][1]))}
               for v in var_names}
        start_x = {v: sp[v]["start"] for v in var_names}
        obj_copy = copy.deepcopy(obj)
        opti = beamOptimizer(bl, p)
        t0   = time.perf_counter()

        try:
            res  = opti.calc(method, seg_var, sp, obj_copy, jac=jac,␣
 ↪options=options,
                             printResults=False, plotProgress=False)
            wall = time.perf_counter() - t0

            result_x_dict = dict(zip(var_names, res.x))
            i_1 = result_x_dict.get("I", np.nan)
            i_3 = result_x_dict.get("I2", np.nan)

            def safe_get_measured(idx):
                if idx in opti.objectives and len(opti.objectives[idx]) > 0:
                    val = opti.objectives[idx][0].get("measured", np.nan)
                    return float(val.item() if hasattr(val, 'item') else val)
                return np.nan

            alpha_x = safe_get_measured(8)
            alpha_y = safe_get_measured(9)

            results.append({
                "run": run_i+1, "method": method_name, "scenario":␣
 ↪scenario_name,
                "final_mse": float(res.fun),
                "nfev": int(getattr(res,'nfev',len(opti.plotMSE))),
```

```
                "nit": int(getattr(res,'nit',-1)),
                "wall_time": round(wall,3),
                "mse_curve": list(opti.plotMSE), "success": bool(res.success),
                "start_x": start_x, "result_x": result_x_dict,
                "I_1": i_1, "I_3": i_3, "alpha_x": alpha_x, "alpha_y": alpha_y
            })
        except Exception as exc:
            wall = time.perf_counter() - t0
            results.append({
                "run": run_i+1, "method": method_name, "scenario":␣
 ↪scenario_name,
                "final_mse": float("nan"), "nfev": -1, "nit": -1,
                "wall_time": round(wall,3), "mse_curve": [], "success": False,
                "start_x": start_x, "result_x": {}, "error": str(exc),
                "I_1": np.nan, "I_3": np.nan, "alpha_x": np.nan, "alpha_y": np.
 ↪nan
            })
            if verbose: print(f"  {method_name} run {run_i+1} FAILED: {exc}")
            continue
        if verbose:
            print(f"  {method_name:22s}  run {run_i+1}/{n_runs}  "
                  f"MSE={results[-1]['final_mse']:.3e}  nit={results[-1]['nit']:
 ↪<3} "
                  f"nfev={results[-1]['nfev']:<4} t={wall:.2f}s")
    return results

def results_to_df(results_list):
    keep = ('run','method','scenario','method_tag','final_mse','nfev','nit',
            'wall_time','success', 'I_1', 'I_3', 'alpha_x', 'alpha_y')
    return pd.DataFrame([{k: r[k] for k in keep if k in r} for r in␣
 ↪results_list])

def plot_stat_convergence(results_by_tag, title="Convergence", figsize=None,
                          colors=None, ax=None, save_path=None, scale="log",
                          convergence_epsilon=None):
    standalone = ax is None
    if standalone: fig, ax = plt.subplots(figsize=figsize or (7, 4.5))
    else: fig = ax.get_figure()

    try:
        cmap = plt.colormaps.get_cmap("tab10")
    except AttributeError:
        cmap = plt.get_cmap("tab10")

    if colors is None: colors = [cmap(i) for i in range(len(results_by_tag))]

    legend_handles = []
```

```
    for (tag, res_list), col in zip(results_by_tag.items(), colors):
        n_t = len(res_list)

        if convergence_epsilon is None:
            converged_iter = [r for r in res_list if r.get('mse_curve')]
        else:
            converged_iter = [r for r in res_list
                    if r.get('mse_curve')
                    and np.isfinite(r['mse_curve'][-1])
                    and r['mse_curve'][-1] < convergence_epsilon]

        n_s = len(converged_iter)
        curves = [r['mse_curve'] for r in converged_iter]

        if not curves:
            legend_handles.append(Line2D([0], [0], color=col, linewidth=2,
                                         label=f"{tag}  (0/{n_t} converged)"))
            continue

        max_len = max(len(c) for c in curves)
        padded  = np.array([c + [c[-1]] * (max_len - len(c)) for c in curves],␣
↪dtype=np.float64)

        padded = np.clip(padded, a_min=1e-20, a_max=1e100)

        evals = np.arange(1, max_len + 1)

        log_padded = np.log10(padded)
        log_mean   = np.mean(log_padded, axis=0)
        log_std    = np.std(log_padded,  axis=0)
        mean       = 10 ** log_mean
        band_lower = 10 ** (log_mean - log_std)
        band_upper = 10 ** (log_mean + log_std)

        ax.plot(evals, mean, color=col, linewidth=2.0, zorder=3)
        ax.fill_between(evals, np.maximum(band_lower, 1e-20), band_upper,
                        color=col, alpha=0.20, zorder=2)

        legend_handles.append(Line2D([0], [0], color=col, linewidth=2,
                                     label=f"{tag}  ({n_s}/{n_t} converged)"))

    ax.set_yscale(scale)
    ax.set_xlabel('Function evaluations (Forward Passes)', fontsize=11)
    ax.set_ylabel('Normalised MSE', fontsize=11)
    ax.set_title(title, fontsize=12, fontweight='bold')
    ax.legend(handles=legend_handles, fontsize=8, framealpha=0.9, loc='upper␣
↪right')
```

```
    ax.grid(True, which='both', alpha=0.25)
    ax.tick_params(labelsize=9)
    if standalone:
        fig.tight_layout()
        if save_path: fig.savefig(save_path, dpi=200, bbox_inches='tight')
        plt.show()
    return ax
print("Benchmark helpers loaded (Warnings Suppressed).")
```

```
The autotime extension is already loaded. To reload it, use:
  %reload_ext autotime
Benchmark helpers loaded (Warnings Suppressed).
time: 16 ms (started: 2026-06-07 21:25:10 -10:00)
```

0.4 4 — Scenario A: Doublet (2 quadrupoles)

Exact replication of Stage 1 from `beamline_optimization.ipynb`: _x = _y = 0 at elements 8–9 (s 0.73–1.0 m).

Simple 2D problem — validates all methods and compares FELsim vs COSY timing.

[17]:
```
def run_scenario_A(CURRENT_BOUNDS, EPSILON, N_RUNS_A, REF_I, A_OBJ, A_VARS,␣
↪A_BEAMLINE_LEN, METHODS_A, SEED=42, scale="log", options=None,␣
↪METHOD_OPTIONS={}):
    A_BOUNDS      = {"I": CURRENT_BOUNDS, "I2": CURRENT_BOUNDS}
    REF_I_1 = REF_I[0]
    REF_I_3 = REF_I[1]
    results_A = {}
    for method_spec in METHODS_A:
        if len(method_spec) == 3:
            method, jac, label = method_spec
        else:
            method, jac = method_spec
            label = None
        options = METHOD_OPTIONS.get(method_label(method, label), None)
        method_name = method_label(method, label)
        tag = method_name + (r" $+\nabla f$" if jac else "")
        if method_name=="trust-constr":
            tag = method_name + r" $+\nabla f+\nabla^2 f$"
        try:
            from IPython.display import Markdown, display
            display(Markdown(f"### Scenario A - {tag}  ({N_RUNS_A} runs)"))
        except Exception:
            print(f"\n Scenario A - {tag}  ({N_RUNS_A} runs)")

        # Execute benchmark (Note: passing 'jac' here prepares for PyTorch␣
↪analytical gradients later)
```

```
        res = run_benchmark(
            "A",
            A_BEAMLINE_LEN,
            PARTICLES,
            A_VARS,
            A_OBJ,
            A_BOUNDS,
            method=method,
            n_runs=N_RUNS_A,
            SEED=SEED,
            seed_offset=0,
            jac=jac,
            options=options,
            method_tag=method_name,
        )
        for r in res:
            r["method_tag"] = tag
        results_A[tag] = res

    # Flatten and merge nested results into a single DataFrame
    df_A = results_to_df([r for v in results_A.values() for r in v])

    #␣
↪============================================================================
    #  CORE FIX: Patch the bug where derivative-free algorithms return␣
↪negative iterations
    #␣
↪============================================================================
    # Since derivative-free algorithms lack internal line searches, their number
    # of function evaluations (nfev) is mathematically equivalent to their true␣
↪exploration steps (nit).
    # If iteration count < 0 is detected, directly extract 'nfev' as the true␣
↪iteration count!
    if 'nit' in df_A.columns and 'nfev' in df_A.columns:
        df_A.loc[df_A['nit'] < 0, 'nit'] = df_A.loc[df_A['nit'] < 0, 'nfev']

    print("\n" + "="*115)
    print(f"  SCENARIO A: Robustness (Tol < {EPSILON}) & Discovered Physics␣
↪State vs Reference")
    print("="*115)

    # 1. Core Evaluation: Check if each run successfully converged below the␣
↪target MSE threshold
    df_A['is_converged'] = df_A['final_mse'] < EPSILON

    # 2. Calculate average performance metrics
```

```
    def _geom_mean_converged(s):
        conv = s[s < EPSILON]
        return 10 ** np.log10(conv).mean() if len(conv) else np.nan
    # ( Note: The redundant 'nfev_mean' has been completely removed as␣
↪requested)
    summary_stats = df_A.groupby('method_tag').agg(
        conv_rate=('is_converged', 'mean'),
        nit_mean=('nit', 'mean'),
        nfev_mean=('nfev', 'mean'),
        wall_time_mean=('wall_time', 'mean'),
    ).join(
        df_A.groupby('method_tag')['final_mse']
            .apply(_geom_mean_converged)
            .rename('final_mse_mean')
    )

    # 3. Extract the Champion Solution: Find the run with the lowest error for␣
↪each method,
    #    extracting its exact hardware currents and resulting alpha observables.
    idx_best = df_A.groupby('method_tag')['final_mse'].idxmin()
    best_runs = df_A.loc[idx_best, ['method_tag', 'I_1', 'I_3', 'alpha_x',␣
↪'alpha_y']].set_index('method_tag')

    # 4. Merge all statistical data
    final_summary = summary_stats.join(best_runs).reset_index()

    # Inject the expert's gold-standard values to establish the Ground Truth
    final_summary['I_1_ref'] = REF_I_1
    final_summary['I_3_ref'] = REF_I_3

    # Reorder columns: Place the AI-predicted currents right next to the␣
↪expert's Reference values
    final_summary = final_summary[[
        'method_tag', 'conv_rate', 'nit_mean', 'nfev_mean', 'wall_time_mean',
        'final_mse_mean', 'I_1_ref', 'I_1', 'I_3_ref', 'I_3', 'alpha_x',␣
↪'alpha_y'
    ]]

    # 5. Plain text formatted output to bypass Jupyter HTML rendering bugs␣
↪(missing tables)
    format_dict = {
        'conv_rate': '{:.0%}',
        'nit_mean': '{:.1f}',
        'nfev_mean': '{:.1f}',
        'wall_time_mean': '{:.3f} s',
        'final_mse_mean': '{:.2e}',
```

```
        'I_1_ref': '{:.4f} A',
        'I_1': '{:.4f} A',
        'I_3_ref': '{:.4f} A',
        'I_3': '{:.4f} A',
        'alpha_x': '{:.2e}',
        'alpha_y': '{:.2e}'
    }

    formatted_df = final_summary.copy()

    # Apply formatting conversions in bulk
    for col, fmt in format_dict.items():
        if col in formatted_df.columns:
            formatted_df[col] = formatted_df[col].apply(lambda x: fmt.format(x)␣
↪if pd.notna(x) else "NaN")

    # Print the final beautifully aligned data table
    print(formatted_df.to_string(index=False, justify='center'))
    print("="*115)

    # Plot the final algorithm stability curve
    plot_stat_convergence(
        results_A,
        title=f"Scenario A: Convergence Stability (Target MSE < {EPSILON})",
        convergence_epsilon=EPSILON,
        scale=scale
    )
    return results_A, df_A, final_summary
```

time: 16 ms (started: 2026-06-07 21:42:55 -10:00)

[8]:
```
%load_ext autotime
def display_results(results_A, REF_I):

    #   Scenario A - Best optimised quadrupole currents per method
    print("  Scenario A: Best Optimised Quadrupole Currents per Method  ")
    print(f"{'Method':<22} {'Best MSE':>12} {'I (A)':>10} {'I2 (A)':>10}")
    print(" " * 58)
    for tag, res_list in results_A.items():
        valid = [r for r in res_list
                 if r.get('result_x') and not np.isnan(r['final_mse'])]
        if not valid:
            print(f"  {tag:<20}  no valid run")
            continue
        best = min(valid, key=lambda r: r['final_mse'])
        I  = best['result_x'].get('I',  float('nan'))
        I2 = best['result_x'].get('I2', float('nan'))
```

```
        print(f"  {tag:<20} {best['final_mse']:>12.4e} {I:>10.4f} {I2:>10.4f}")
    print()
    print(f"  Reference FELsim S1 currents: I={REF_I[0]:.4f} A,  I2={REF_I[1]:.
↪4f} A")
    # Mean best across all runs
    all_valid = [r for v in results_A.values() for r in v
                 if r.get('result_x') and not np.isnan(r['final_mse'])]
    if all_valid:
        best_overall = min(all_valid, key=lambda r: r['final_mse'])
        print(f"\n  Overall best across all methods/runs:")
        print(f"    Method: {best_overall['method_tag']}, ␣
↪MSE={best_overall['final_mse']:.4e}")
        print(f"    I={best_overall['result_x']['I']:.5f} A,  "
            f"I2={best_overall['result_x']['I2']:.5f} A")
```

```
The autotime extension is already loaded. To reload it, use:
  %reload_ext autotime
time: 16 ms (started: 2026-06-07 21:25:10 -10:00)
```

[9]:
```
# check default setting for each method
import inspect

import scipy.optimize._lbfgsb_py as lbfgsb
import scipy.optimize._optimize as optimize_core
import scipy.optimize._slsqp_py as slsqp
import scipy.optimize._cobyla_py as cobyla
from scipy.optimize._trustregion_constr import minimize_trustregion_constr as␣
↪trust_constr

MINIMIZER_FUNCS = {
    "L-BFGS-B": lbfgsb._minimize_lbfgsb,
    "Nelder-Mead": optimize_core._minimize_neldermead,
    "SLSQP": slsqp._minimize_slsqp,
    "trust-constr": trust_constr._minimize_trustregion_constr,
    "COBYLA": cobyla._minimize_cobyla,
}

for method_name, minimize_func in MINIMIZER_FUNCS.items():
    print(f"\n## {method_name}")
    print(minimize_func.__doc__)
    print(inspect.signature(minimize_func))
```

```
## L-BFGS-B

    Minimize a scalar function of one or more variables using the L-BFGS-B
    algorithm.
```

```
Options
-------
disp : None or int
    Deprecated option that previously controlled the text printed on the
    screen during the problem solution. Now the code does not emit any
    output and this keyword has no function.

    .. deprecated:: 1.15.0
        This keyword is deprecated and will be removed from SciPy 1.18.0.

maxcor : int
    The maximum number of variable metric corrections used to
    define the limited memory matrix. (The limited memory BFGS
    method does not store the full hessian but uses this many terms
    in an approximation to it.)
ftol : float
    The iteration stops when ``(f^k -
    f^{k+1})/max{|f^k|,|f^{k+1}|,1} <= ftol``.
gtol : float
    The iteration will stop when ``max{|proj g_i | i = 1, …, n}
    <= gtol`` where ``proj g_i`` is the i-th component of the
    projected gradient.
eps : float or ndarray
    If `jac is None` the absolute step size used for numerical
    approximation of the jacobian via forward differences.
maxfun : int
    Maximum number of function evaluations before minimization terminates.
    Note that this function may violate the limit if the gradients
    are evaluated by numerical differentiation.
maxiter : int
    Maximum number of algorithm iterations.
iprint : int, optional
    Deprecated option that previously controlled the text printed on the
    screen during the problem solution. Now the code does not emit any
    output and this keyword has no function.

    .. deprecated:: 1.15.0
        This keyword is deprecated and will be removed from SciPy 1.18.0.

maxls : int, optional
    Maximum number of line search steps (per iteration). Default is 20.
finite_diff_rel_step : None or array_like, optional
    If ``jac in ['2-point', '3-point', 'cs']`` the relative step size to
    use for numerical approximation of the jacobian. The absolute step
    size is computed as ``h = rel_step * sign(x) * max(1, abs(x))``,
    possibly adjusted to fit into the bounds. For ``method='3-point'``
    the sign of `h` is ignored. If None (default) then step is selected
    automatically.
```

```
    workers : int, map-like callable, optional
        A map-like callable, such as `multiprocessing.Pool.map` for evaluating
        any numerical differentiation in parallel.
        This evaluation is carried out as ``workers(fun, iterable)``.

        .. versionadded:: 1.16.0

    Notes
    -----
    The option `ftol` is exposed via the `scipy.optimize.minimize` interface,
    but calling `scipy.optimize.fmin_l_bfgs_b` directly exposes `factr`. The
    relationship between the two is ``ftol = factr * numpy.finfo(float).eps``.
    I.e., `factr` multiplies the default machine floating-point precision to
    arrive at `ftol`.
    If the minimization is slow to converge the optimizer may halt if the
    total number of function evaluations exceeds `maxfun`, or the number of
    algorithm iterations has reached `maxiter` (whichever comes first). If
    this is the case then ``result.success=False``, and an appropriate
    error message is contained in ``result.message``.


(fun, x0, args=(), jac=None, bounds=None, disp=<object object at
0x000002841A401A50>, maxcor=10, ftol=2.220446049250313e-09, gtol=1e-05,
eps=1e-08, maxfun=15000, maxiter=15000, iprint=<object object at
0x000002841A401A50>, callback=None, maxls=20, finite_diff_rel_step=None,
workers=None, **unknown_options)

## Nelder-Mead

    Minimization of scalar function of one or more variables using the
    Nelder-Mead algorithm.

    Options
    -------
    disp : bool
        Set to True to print convergence messages.
    maxiter, maxfev : int
        Maximum allowed number of iterations and function evaluations.
        Will default to ``N*200``, where ``N`` is the number of
        variables, if neither `maxiter` or `maxfev` is set. If both
        `maxiter` and `maxfev` are set, minimization will stop at the
        first reached.
    return_all : bool, optional
        Set to True to return a list of the best solution at each of the
        iterations.
    initial_simplex : array_like of shape (N + 1, N)
        Initial simplex. If given, overrides `x0`.
        ``initial_simplex[j,:]`` should contain the coordinates of
```

```
        the jth vertex of the ``N+1`` vertices in the simplex, where
        ``N`` is the dimension.
    xatol : float, optional
        Absolute error in xopt between iterations that is acceptable for
        convergence.
    fatol : number, optional
        Absolute error in func(xopt) between iterations that is acceptable for
        convergence.
    adaptive : bool, optional
        Adapt algorithm parameters to dimensionality of problem. Useful for
        high-dimensional minimization [1]_.
    bounds : sequence or `Bounds`, optional
        Bounds on variables. There are two ways to specify the bounds:

        1. Instance of `Bounds` class.
        2. Sequence of ``(min, max)`` pairs for each element in `x`. None
           is used to specify no bound.

        Note that this just clips all vertices in simplex based on
        the bounds.

    References
    ----------
    .. [1] Gao, F. and Han, L.
       Implementing the Nelder-Mead simplex algorithm with adaptive
       parameters. 2012. Computational Optimization and Applications.
       51:1, pp. 259-277


(func, x0, args=(), callback=None, maxiter=None, maxfev=None, disp=False,
return_all=False, initial_simplex=None, xatol=0.0001, fatol=0.0001,
adaptive=False, bounds=None, **unknown_options)

## SLSQP

    Minimize a scalar function of one or more variables using Sequential
    Least Squares Programming (SLSQP).

    Parameters
    ----------
    ftol : float
        Precision target for the value of f in the stopping criterion. This
value
        controls the final accuracy for checking various optimality conditions;
        gradient of the lagrangian and absolute sum of the constraint violations
        should be lower than ``ftol``. Similarly, computed step size and the
        objective function changes are checked against this value. Default is
1e-6.
```

```
    eps : float
        Step size used for numerical approximation of the Jacobian.
    disp : bool
        Set to True to print convergence messages. If False,
        `verbosity` is ignored and set to 0.
    maxiter : int
        Maximum number of iterations.
    finite_diff_rel_step : None or array_like, optional
        If ``jac in ['2-point', '3-point', 'cs']`` the relative step size to
        use for numerical approximation of `jac`. The absolute step
        size is computed as ``h = rel_step * sign(x) * max(1, abs(x))``,
        possibly adjusted to fit into the bounds. For ``method='3-point'``
        the sign of `h` is ignored. If None (default) then step is selected
        automatically.
    workers : int, map-like callable, optional
        A map-like callable, such as `multiprocessing.Pool.map` for evaluating
        any numerical differentiation in parallel.
        This evaluation is carried out as ``workers(fun, iterable)``.

        .. versionadded:: 1.16.0

    Returns
    -------
    res : OptimizeResult
        The optimization result represented as an `OptimizeResult` object.
        In this dict-like object the following fields are of particular
importance:
        ``x`` the solution array, ``success`` a Boolean flag indicating if the
        optimizer exited successfully, ``message`` which describes the reason
for
        termination, and ``multipliers`` which contains the Karush-Kuhn-Tucker
        (KKT) multipliers for the QP approximation used in solving the original
        nonlinear problem. See ``Notes`` below. See also `OptimizeResult` for a
        description of other attributes.

    Notes
    -----
    The KKT multipliers are returned in the ``OptimizeResult.multipliers``
    attribute as a NumPy array. Denoting the dimension of the equality
constraints
    with ``meq``, and of inequality constraints with ``mineq``, then the
returned
    array slice ``m[:meq]`` contains the multipliers for the equality
constraints,
    and the remaining ``m[meq:meq + mineq]`` contains the multipliers for the
    inequality constraints. The multipliers corresponding to bound inequalities
    are not returned. See [1]_ pp. 321 or [2]_ for an explanation of how to
interpret
```

```
    these multipliers. The internal QP problem is solved using the methods given
    in [3]_ Chapter 25.

    Note that if new-style `NonlinearConstraint` or `LinearConstraint` were
    used, then ``minimize`` converts them first to old-style constraint dicts.
    It is possible for a single new-style constraint to simultaneously contain
    both inequality and equality constraints. This means that if there is mixing
    within a single constraint, then the returned list of multipliers will have
    a different length than the original new-style constraints.

    References
    ----------
    .. [1] Nocedal, J., and S J Wright, 2006, "Numerical Optimization",
Springer,
       New York.
    .. [2] Kraft, D., "A software package for sequential quadratic programming",
       1988, Tech. Rep. DFVLR-FB 88-28, DLR German Aerospace Center, Germany.
    .. [3] Lawson, C. L., and R. J. Hanson, 1995, "Solving Least Squares
Problems",
       SIAM, Philadelphia, PA.


(func, x0, args=(), jac=None, bounds=None, constraints=(), maxiter=100,
ftol=1e-06, iprint=1, disp=False, eps=np.float64(1.4901161193847656e-08),
callback=None, finite_diff_rel_step=None, workers=None, **unknown_options)

## trust-constr
Minimize a scalar function subject to constraints.

    Parameters
    ----------
    gtol : float, optional
        Tolerance for termination by the norm of the Lagrangian gradient.
        The algorithm will terminate when both the infinity norm (i.e., max
        abs value) of the Lagrangian gradient and the constraint violation
        are smaller than ``gtol``. Default is 1e-8.
    xtol : float, optional
        Tolerance for termination by the change of the independent variable.
        The algorithm will terminate when ``tr_radius < xtol``, where
        ``tr_radius`` is the radius of the trust region used in the algorithm.
        Default is 1e-8.
    barrier_tol : float, optional
        Threshold on the barrier parameter for the algorithm termination.
        When inequality constraints are present, the algorithm will terminate
        only when the barrier parameter is less than `barrier_tol`.
        Default is 1e-8.
    sparse_jacobian : {bool, None}, optional
        Determines how to represent Jacobians of the constraints. If bool,
```

```
        then Jacobians of all the constraints will be converted to the
        corresponding format. If None (default), then Jacobians won't be
        converted, but the algorithm can proceed only if they all have the
        same format.
    initial_tr_radius: float, optional
        Initial trust radius. The trust radius gives the maximum distance
        between solution points in consecutive iterations. It reflects the
        trust the algorithm puts in the local approximation of the optimization
        problem. For an accurate local approximation the trust-region should be
        large and for an  approximation valid only close to the current point it
        should be a small one. The trust radius is automatically updated
throughout
        the optimization process, with ``initial_tr_radius`` being its initial
value.
        Default is 1 (recommended in [1]_, p. 19).
    initial_constr_penalty : float, optional
        Initial constraints penalty parameter. The penalty parameter is used for
        balancing the requirements of decreasing the objective function
        and satisfying the constraints. It is used for defining the merit
function:
        ``merit_function(x) = fun(x) + constr_penalty * constr_norm_l2(x)``,
        where ``constr_norm_l2(x)`` is the l2 norm of a vector containing all
        the constraints. The merit function is used for accepting or rejecting
        trial points and ``constr_penalty`` weights the two conflicting goals
        of reducing objective function and constraints. The penalty is
automatically
        updated throughout the optimization  process, with
        ``initial_constr_penalty`` being its  initial value. Default is 1
        (recommended in [1]_, p 19).
    initial_barrier_parameter, initial_barrier_tolerance: float, optional
        Initial barrier parameter and initial tolerance for the barrier
subproblem.
        Both are used only when inequality constraints are present. For dealing
with
        optimization problems ``min_x f(x)`` subject to inequality constraints
        ``c(x) <= 0`` the algorithm introduces slack variables, solving the
problem
        ``min_(x,s) f(x) + barrier_parameter*sum(ln(s))`` subject to the
equality
        constraints  ``c(x) + s = 0`` instead of the original problem. This
subproblem
        is solved for decreasing values of ``barrier_parameter`` and with
decreasing
        tolerances for the termination, starting with
``initial_barrier_parameter``
        for the barrier parameter and ``initial_barrier_tolerance`` for the
        barrier tolerance. Default is 0.1 for both values (recommended in [1]_
p. 19).
```

```
        Also note that ``barrier_parameter`` and ``barrier_tolerance`` are
updated
        with the same prefactor.
    factorization_method : string or None, optional
        Method to factorize the Jacobian of the constraints. Use None (default)
        for the auto selection or one of:

        - 'NormalEquation' (requires scikit-sparse)
        - 'AugmentedSystem'
        - 'QRFactorization'
        - 'SVDFactorization'

        The methods 'NormalEquation' and 'AugmentedSystem' can be used only
        with sparse constraints. The projections required by the algorithm
        will be computed using, respectively, the normal equation  and the
        augmented system approaches explained in [1]_. 'NormalEquation'
        computes the Cholesky factorization of ``A A.T`` and 'AugmentedSystem'
        performs the LU factorization of an augmented system. They usually
        provide similar results. 'AugmentedSystem' is used by default for
        sparse matrices.

        The methods 'QRFactorization' and 'SVDFactorization' can be used
        only with dense constraints. They compute the required projections
        using, respectively, QR and SVD factorizations. The 'SVDFactorization'
        method can cope with Jacobian matrices with deficient row rank and will
        be used whenever other factorization methods fail (which may imply the
        conversion of sparse matrices to a dense format when required).
        By default, 'QRFactorization' is used for dense matrices.
    finite_diff_rel_step : None or array_like, optional
        Relative step size for the finite difference approximation.
    maxiter : int, optional
        Maximum number of algorithm iterations. Default is 1000.
    verbose : {0, 1, 2, 3}, optional
        Level of algorithm's verbosity:

        * 0 (default) : work silently.
        * 1 : display a termination report.
        * 2 : display progress during iterations.
        * 3 : display progress during iterations (more complete report).

    disp : bool, optional
        If True (default), then `verbose` will be set to 1 if it was 0.
    workers : int, map-like callable, optional
        A map-like callable, such as `multiprocessing.Pool.map` for evaluating
        any numerical differentiation in parallel.
        This evaluation is carried out as ``workers(fun, iterable)``.

        .. versionadded:: 1.16.0
```

```
Returns
-------
`OptimizeResult` with the fields documented below. Note the following:

    1. All values corresponding to the constraints are ordered as they
       were passed to the solver. And values corresponding to `bounds`
       constraints are put *after* other constraints.
    2. All numbers of function, Jacobian or Hessian evaluations correspond
       to numbers of actual Python function calls. It means, for example,
       that if a Jacobian is estimated by finite differences, then the
       number of Jacobian evaluations will be zero and the number of
       function evaluations will be incremented by all calls during the
       finite difference estimation.

x : ndarray, shape (n,)
    Solution found.
optimality : float
    Infinity norm of the Lagrangian gradient at the solution.
constr_violation : float
    Maximum constraint violation at the solution.
fun : float
    Objective function at the solution.
grad : ndarray, shape (n,)
    Gradient of the objective function at the solution.
lagrangian_grad : ndarray, shape (n,)
    Gradient of the Lagrangian function at the solution.
nit : int
    Total number of iterations.
nfev : integer
    Number of the objective function evaluations.
njev : integer
    Number of the objective function gradient evaluations.
nhev : integer
    Number of the objective function Hessian evaluations.
cg_niter : int
    Total number of the conjugate gradient method iterations.
method : {'equality_constrained_sqp', 'tr_interior_point'}
    Optimization method used.
constr : list of ndarray
    List of constraint values at the solution.
jac : list of {ndarray, sparse array}
    List of the Jacobian matrices of the constraints at the solution.
v : list of ndarray
    List of the Lagrange multipliers for the constraints at the solution.
    For an inequality constraint a positive multiplier means that the upper
    bound is active, a negative multiplier means that the lower bound is
    active and if a multiplier is zero it means the constraint is not
```

```
    active.
constr_nfev : list of int
    Number of constraint evaluations for each of the constraints.
constr_njev : list of int
    Number of Jacobian matrix evaluations for each of the constraints.
constr_nhev : list of int
    Number of Hessian evaluations for each of the constraints.
tr_radius : float
    Radius of the trust region at the last iteration.
constr_penalty : float
    Penalty parameter at the last iteration, see `initial_constr_penalty`.
barrier_tolerance : float
    Tolerance for the barrier subproblem at the last iteration.
    Only for problems with inequality constraints.
barrier_parameter : float
    Barrier parameter at the last iteration. Only for problems
    with inequality constraints.
execution_time : float
    Total execution time.
message : str
    Termination message.
status : {0, 1, 2, 3, 4}
    Termination status:

    * 0 : The maximum number of function evaluations is exceeded.
    * 1 : `gtol` termination condition is satisfied.
    * 2 : `xtol` termination condition is satisfied.
    * 3 : `callback` function requested termination.
    * 4 : Constraint violation exceeds 'gtol'.

    .. versionchanged:: 1.15.0
        If the constraint violation exceeds `gtol`, then ``result.success``
        will now be False.

cg_stop_cond : int
    Reason for CG subproblem termination at the last iteration:

    * 0 : CG subproblem not evaluated.
    * 1 : Iteration limit was reached.
    * 2 : Reached the trust-region boundary.
    * 3 : Negative curvature detected.
    * 4 : Tolerance was satisfied.

References
----------
.. [1] Conn, A. R., Gould, N. I., & Toint, P. L.
       Trust region methods. 2000. Siam. pp. 19.
```

```
(fun, x0, args, grad, hess, hessp, bounds, constraints, xtol=1e-08, gtol=1e-08,
barrier_tol=1e-08, sparse_jacobian=None, callback=None, maxiter=1000, verbose=0,
finite_diff_rel_step=None, initial_constr_penalty=1.0, initial_tr_radius=1.0,
initial_barrier_parameter=0.1, initial_barrier_tolerance=0.1,
factorization_method=None, disp=False, workers=None)

## COBYLA

    Minimize a scalar function of one or more variables using the
    Constrained Optimization BY Linear Approximation (COBYLA) algorithm.
    This method uses the pure-python implementation of the algorithm from PRIMA.

    Options
    -------
    rhobeg : float
        Reasonable initial changes to the variables.
    tol : float
        Final accuracy in the optimization (not precisely guaranteed).
        This is a lower bound on the size of the trust region.
    disp : int
        Controls the frequency of output:
            0. (default) There will be no printing
            1. A message will be printed to the screen at the end of iteration,
showing
               the best vector of variables found and its objective function
value
            2. in addition to 1, each new value of RHO is printed to the screen,
               with the best vector of variables so far and its objective
function
               value.
            3. in addition to 2, each function evaluation with its variables
will
               be printed to the screen.
    maxiter : int
        Maximum number of function evaluations.
    catol : float
        Tolerance (absolute) for constraint violations
    f_target : float
        Stop if the objective function is less than `f_target`.

        .. versionchanged:: 1.16.0
            The original Powell implementation was replaced by a pure
            Python version from the PRIMA package, with bug fixes and
            improvements being made.


    References
    ----------
```

```
    Zhang Z. (2023), "PRIMA: Reference Implementation for Powell's Methods with
    Modernization and Amelioration", https://www.libprima.net,
    :doi:`10.5281/zenodo.8052654`

(fun, x0, args=(), constraints=(), rhobeg=1.0, tol=0.0001, maxiter=1000, disp=0,
catol=None, f_target=-inf, callback=None, bounds=None, **unknown_options)
time: 0 ns (started: 2026-06-07 21:25:10 -10:00)
```

[20]:
```
CURRENT_BOUNDS = (0.01, 1.5)
EPSILON = 1e-3

REF_I_1 = 0.8218
REF_I_3 = 1.0430
REF_I = [REF_I_1, REF_I_3]

# parameter setting
N_RUNS_A = 100
A_BEAMLINE_LEN = 10

# this is default version
METHOD_OPTIONS = {
    "L-BFGS-B": {
        "maxcor": 10,
        "ftol": 2.220446049250313e-09,
        "gtol": 1e-5,
        "eps": 1e-8,
        "maxfun": 15000,
        "maxiter": 15000,
        "maxls": 20,
    },
    "SLSQP": {
        "maxiter": 100,
        "ftol": 1e-6,
        "iprint": 1,
        "disp": False,
        "eps": 1.4901161193847656e-08,
        "finite_diff_rel_step": None,
        "workers": None,
    },
    "trust-constr": {
        "xtol": 1e-8,
        "gtol": 1e-8,
        "barrier_tol": 1e-8,
        "sparse_jacobian": None,
        "maxiter": 1000,
        "verbose": 0,
        "finite_diff_rel_step": None,
```

```
        "initial_constr_penalty": 1.0,
        "initial_tr_radius": 1.0,
        "initial_barrier_parameter": 0.1,
        "initial_barrier_tolerance": 0.1,
        "factorization_method": None,
        "disp": False,
        "workers": None,
    },
    "Nelder-Mead": {
        "maxiter": None,
        "maxfev": None,
        "disp": False,
        "return_all": False,
        "initial_simplex": None,
        "xatol": 1e-4,
        "fatol": 1e-4,
        "adaptive": False,
    },
    "COBYLA": {
        "rhobeg": 1.0,
        "tol": 1e-4,
        "maxiter": 1000,
        "disp": 0,
        "catol": None,
        "f_target": float("-inf"),
    },
}

METHODS_A = [
    ("Nelder-Mead", None),
    ("L-BFGS-B", "2-point"),
    ("SLSQP", "2-point"),
    ("COBYLA", None),
    ("trust-constr", None),
]

A_VARS = {
    1: ["I", "current", lambda x: x],
    3: ["I2", "current", lambda x: x],
}
A_OBJ = {
    8: [{"measure": ["x", "alpha"], "goal": 0.0, "weight": 1.0}],
    9: [{"measure": ["y", "alpha"], "goal": 0.0, "weight": 1.0}],
}
```

time: 0 ns (started: 2026-06-07 21:49:16 -10:00)

[11]:
```
# # test whether float64 works
# import numpy as np, copy

# print("PARTICLES dtype:", PARTICLES.dtype)

# A_BEAMLINE_LEN = 138
# bl = ExcelElements(EXCEL_PATH).create_beamline()[:A_BEAMLINE_LEN]
# opti = beamOptimizer(bl, PARTICLES.clone())
# opti.calc("Nelder-Mead", A_VARS,
#           {"I":{"bounds":(0.01,1.5),"start":0.5}, "I2":{"bounds":(0.01,1.
 ↪5),"start":0.5}},
#           copy.deepcopy(A_OBJ), printResults=False)

# print("variablesToOptimize:", opti.variablesToOptimize)

# x = np.array([0.5, 0.5])
# base = opti._optiSpeed(x)
# print(f"\nbase MSE = {base:.6e}")
# print("test:")
# for h in [1e-5, 1e-7, 1e-9, 1e-11]:
#     xp = x.copy(); xp[0] += h
#     diff = opti._optiSpeed(xp) - base
#     print(f"  h={h:.0e}:  ΔMSE={diff:+.3e}   slope {diff/h:+.4e}")
```

```
time: 0 ns (started: 2026-06-07 21:25:10 -10:00)
```

[21]:
```
%load_ext autotime

results_A, df_A, final_summary = run_scenario_A(CURRENT_BOUNDS, EPSILON,␣
 ↪N_RUNS_A, REF_I, A_OBJ, A_VARS, A_BEAMLINE_LEN,␣
 ↪METHODS_A,scale="log",METHOD_OPTIONS=METHOD_OPTIONS)
display_results(results_A, REF_I)
```

```
The autotime extension is already loaded. To reload it, use:
  %reload_ext autotime
```

0.4.1 Scenario A — Nelder-Mead (100 runs)

```
  Nelder-Mead            run 1/100   MSE=7.390e-09  nit=36  nfev=69   t=13.44s
  Nelder-Mead            run 2/100   MSE=2.273e-09  nit=32  nfev=61   t=13.07s
  Nelder-Mead            run 3/100   MSE=6.599e-09  nit=54  nfev=103  t=23.57s
  Nelder-Mead            run 4/100   MSE=8.404e-09  nit=39  nfev=71   t=27.27s
  Nelder-Mead            run 5/100   MSE=1.976e-09  nit=40  nfev=75   t=21.68s
  Nelder-Mead            run 6/100   MSE=6.858e-08  nit=24  nfev=48   t=10.03s
  Nelder-Mead            run 7/100   MSE=1.374e-08  nit=37  nfev=72   t=14.43s
  Nelder-Mead            run 8/100   MSE=6.627e-09  nit=54  nfev=102  t=22.51s
  Nelder-Mead            run 9/100   MSE=6.049e-09  nit=47  nfev=88   t=18.33s
  Nelder-Mead            run 10/100  MSE=5.580e-09  nit=47  nfev=88   t=19.53s
  Nelder-Mead            run 11/100  MSE=5.432e-09  nit=39  nfev=77   t=16.60s
```

```
Nelder-Mead            run 12/100  MSE=4.470e-09  nit=46  nfev=85   t=18.53s
Nelder-Mead            run 13/100  MSE=1.990e-09  nit=49  nfev=93   t=20.54s
Nelder-Mead            run 14/100  MSE=3.455e-09  nit=34  nfev=65   t=14.03s
Nelder-Mead            run 15/100  MSE=9.621e-09  nit=32  nfev=59   t=11.85s
Nelder-Mead            run 16/100  MSE=2.798e-09  nit=37  nfev=70   t=15.47s
Nelder-Mead            run 17/100  MSE=5.633e-09  nit=44  nfev=83   t=18.78s
Nelder-Mead            run 18/100  MSE=6.589e-09  nit=44  nfev=84   t=17.09s
Nelder-Mead            run 19/100  MSE=3.123e-09  nit=31  nfev=59   t=12.01s
Nelder-Mead            run 20/100  MSE=6.214e-09  nit=50  nfev=95   t=23.56s
Nelder-Mead            run 21/100  MSE=7.893e-08  nit=42  nfev=80   t=18.07s
Nelder-Mead            run 22/100  MSE=5.112e-09  nit=38  nfev=71   t=14.38s
Nelder-Mead            run 23/100  MSE=1.461e-09  nit=44  nfev=85   t=17.19s
Nelder-Mead            run 24/100  MSE=6.132e-09  nit=47  nfev=92   t=19.06s
Nelder-Mead            run 25/100  MSE=2.444e-09  nit=48  nfev=93   t=19.06s
Nelder-Mead            run 26/100  MSE=1.509e-08  nit=43  nfev=77   t=15.65s
Nelder-Mead            run 27/100  MSE=9.769e-09  nit=51  nfev=93   t=20.41s
Nelder-Mead            run 28/100  MSE=2.187e-08  nit=44  nfev=79   t=16.68s
Nelder-Mead            run 29/100  MSE=7.937e-09  nit=39  nfev=74   t=15.82s
Nelder-Mead            run 30/100  MSE=1.464e-08  nit=45  nfev=79   t=16.12s
Nelder-Mead            run 31/100  MSE=9.592e-09  nit=34  nfev=64   t=13.02s
Nelder-Mead            run 32/100  MSE=1.132e-09  nit=42  nfev=84   t=17.55s
Nelder-Mead            run 33/100  MSE=2.967e-09  nit=32  nfev=62   t=14.02s
Nelder-Mead            run 34/100  MSE=6.733e-09  nit=47  nfev=87   t=20.06s
Nelder-Mead            run 35/100  MSE=5.206e-09  nit=48  nfev=89   t=18.89s
Nelder-Mead            run 36/100  MSE=5.295e-10  nit=35  nfev=69   t=15.26s
Nelder-Mead            run 37/100  MSE=4.628e-09  nit=44  nfev=82   t=19.15s
Nelder-Mead            run 38/100  MSE=1.317e-08  nit=69  nfev=130  t=28.96s
Nelder-Mead            run 39/100  MSE=2.500e-09  nit=40  nfev=76   t=16.74s
Nelder-Mead            run 40/100  MSE=1.539e+01  nit=26  nfev=50   t=11.12s
Nelder-Mead            run 41/100  MSE=7.911e-09  nit=35  nfev=69   t=15.59s
Nelder-Mead            run 42/100  MSE=1.510e-08  nit=41  nfev=82   t=19.29s
Nelder-Mead            run 43/100  MSE=5.731e-09  nit=39  nfev=76   t=16.39s
Nelder-Mead            run 44/100  MSE=8.586e-02  nit=13  nfev=25   t=5.99s
Nelder-Mead            run 45/100  MSE=3.659e-08  nit=33  nfev=63   t=14.93s
Nelder-Mead            run 46/100  MSE=9.646e-09  nit=32  nfev=62   t=13.72s
Nelder-Mead            run 47/100  MSE=4.137e-08  nit=41  nfev=80   t=17.93s
Nelder-Mead            run 48/100  MSE=6.184e-09  nit=36  nfev=69   t=16.77s
Nelder-Mead            run 49/100  MSE=5.696e-08  nit=34  nfev=66   t=15.89s
Nelder-Mead            run 50/100  MSE=5.131e-09  nit=66  nfev=131  t=29.51s
Nelder-Mead            run 51/100  MSE=5.109e-09  nit=39  nfev=74   t=16.90s
Nelder-Mead            run 52/100  MSE=2.709e-09  nit=34  nfev=63   t=13.73s
Nelder-Mead            run 53/100  MSE=6.033e-09  nit=44  nfev=84   t=20.56s
Nelder-Mead            run 54/100  MSE=4.511e-09  nit=50  nfev=99   t=18.89s
Nelder-Mead            run 55/100  MSE=3.972e-09  nit=51  nfev=96   t=16.39s
Nelder-Mead            run 56/100  MSE=1.190e-08  nit=32  nfev=60   t=9.80s
Nelder-Mead            run 57/100  MSE=1.239e-08  nit=36  nfev=69   t=13.13s
Nelder-Mead            run 58/100  MSE=1.253e-08  nit=48  nfev=90   t=14.11s
Nelder-Mead            run 59/100  MSE=8.114e-09  nit=42  nfev=85   t=16.32s
```

```
Nelder-Mead            run 60/100  MSE=3.689e-09  nit=45  nfev=85   t=14.16s
Nelder-Mead            run 61/100  MSE=1.252e-08  nit=44  nfev=86   t=14.34s
Nelder-Mead            run 62/100  MSE=2.300e-08  nit=39  nfev=74   t=14.37s
Nelder-Mead            run 63/100  MSE=4.794e-09  nit=42  nfev=81   t=18.94s
Nelder-Mead            run 64/100  MSE=9.388e-09  nit=47  nfev=89   t=16.07s
Nelder-Mead            run 65/100  MSE=6.266e-09  nit=51  nfev=93   t=19.40s
Nelder-Mead            run 66/100  MSE=1.487e-09  nit=39  nfev=74   t=17.17s
Nelder-Mead            run 67/100  MSE=2.333e-08  nit=43  nfev=78   t=17.79s
Nelder-Mead            run 68/100  MSE=1.958e-08  nit=30  nfev=58   t=13.74s
Nelder-Mead            run 69/100  MSE=1.539e+01  nit=28  nfev=52   t=11.62s
Nelder-Mead            run 70/100  MSE=8.347e-09  nit=39  nfev=77   t=16.21s
Nelder-Mead            run 71/100  MSE=8.586e-02  nit=16  nfev=31   t=6.16s
Nelder-Mead            run 72/100  MSE=8.586e-02  nit=13  nfev=25   t=6.37s
Nelder-Mead            run 73/100  MSE=5.876e-09  nit=37  nfev=73   t=14.74s
Nelder-Mead            run 74/100  MSE=3.749e-08  nit=40  nfev=77   t=15.15s
Nelder-Mead            run 75/100  MSE=1.425e-09  nit=38  nfev=73   t=14.30s
Nelder-Mead            run 76/100  MSE=1.262e-08  nit=41  nfev=79   t=15.31s
Nelder-Mead            run 77/100  MSE=4.097e-09  nit=38  nfev=72   t=14.03s
Nelder-Mead            run 78/100  MSE=2.677e-08  nit=40  nfev=74   t=13.93s
Nelder-Mead            run 79/100  MSE=8.000e-09  nit=37  nfev=75   t=14.37s
Nelder-Mead            run 80/100  MSE=8.586e-02  nit=13  nfev=25   t=4.90s
Nelder-Mead            run 81/100  MSE=5.683e-09  nit=36  nfev=69   t=13.11s
Nelder-Mead            run 82/100  MSE=1.077e-09  nit=36  nfev=72   t=13.64s
Nelder-Mead            run 83/100  MSE=1.707e-09  nit=35  nfev=66   t=12.29s
Nelder-Mead            run 84/100  MSE=3.031e-09  nit=43  nfev=80   t=15.53s
Nelder-Mead            run 85/100  MSE=2.981e-08  nit=40  nfev=77   t=15.75s
Nelder-Mead            run 86/100  MSE=1.512e-09  nit=40  nfev=77   t=16.41s
Nelder-Mead            run 87/100  MSE=1.074e-08  nit=32  nfev=61   t=11.51s
Nelder-Mead            run 88/100  MSE=8.017e-09  nit=31  nfev=60   t=11.77s
Nelder-Mead            run 89/100  MSE=4.597e-09  nit=35  nfev=65   t=12.47s
Nelder-Mead            run 90/100  MSE=2.956e-09  nit=33  nfev=62   t=11.55s
Nelder-Mead            run 91/100  MSE=5.772e-09  nit=44  nfev=83   t=16.11s
Nelder-Mead            run 92/100  MSE=4.294e-08  nit=27  nfev=53   t=9.98s
Nelder-Mead            run 93/100  MSE=1.804e-08  nit=35  nfev=65   t=12.93s
Nelder-Mead            run 94/100  MSE=2.281e-08  nit=40  nfev=76   t=14.94s
Nelder-Mead            run 95/100  MSE=4.596e-09  nit=38  nfev=73   t=13.84s
Nelder-Mead            run 96/100  MSE=1.278e-08  nit=41  nfev=77   t=14.97s
Nelder-Mead            run 97/100  MSE=9.072e-10  nit=35  nfev=67   t=12.82s
Nelder-Mead            run 98/100  MSE=1.696e-08  nit=42  nfev=81   t=15.50s
Nelder-Mead            run 99/100  MSE=3.522e-09  nit=40  nfev=75   t=14.56s
Nelder-Mead            run 100/100  MSE=2.082e-09  nit=39  nfev=72   t=14.12s
```

0.4.2 Scenario A — L-BFGS-B $+\nabla f$ (100 runs)

```
L-BFGS-B               run 1/100  MSE=3.672e-14  nit=9   nfev=48   t=9.00s
L-BFGS-B               run 2/100  MSE=1.925e-12  nit=12  nfev=57   t=10.77s
L-BFGS-B               run 3/100  MSE=7.128e-13  nit=12  nfev=60   t=12.47s
L-BFGS-B               run 4/100  MSE=1.488e-13  nit=10  nfev=42   t=8.76s
```

```
L-BFGS-B                run 5/100  MSE=4.492e-13  nit=16  nfev=81   t=17.07s
L-BFGS-B                run 6/100  MSE=4.907e-13  nit=8   nfev=36   t=6.93s
L-BFGS-B                run 7/100  MSE=1.226e-12  nit=15  nfev=69   t=13.56s
L-BFGS-B                run 8/100  MSE=1.459e-13  nit=9   nfev=45   t=8.65s
L-BFGS-B                run 9/100  MSE=4.516e-13  nit=11  nfev=42   t=8.18s
L-BFGS-B                run 10/100  MSE=1.246e-13  nit=10  nfev=51   t=9.37s
L-BFGS-B                run 11/100  MSE=1.310e-12  nit=9   nfev=36   t=6.80s
L-BFGS-B                run 12/100  MSE=9.065e-13  nit=11  nfev=42   t=7.80s
L-BFGS-B                run 13/100  MSE=4.530e-11  nit=8   nfev=48   t=9.16s
L-BFGS-B                run 14/100  MSE=5.555e-12  nit=15  nfev=51   t=10.21s
L-BFGS-B                run 15/100  MSE=5.254e-13  nit=11  nfev=48   t=8.95s
L-BFGS-B                run 16/100  MSE=5.105e-13  nit=14  nfev=48   t=9.07s
L-BFGS-B                run 17/100  MSE=4.633e-14  nit=14  nfev=60   t=11.65s
L-BFGS-B                run 18/100  MSE=5.575e-13  nit=13  nfev=63   t=11.82s
L-BFGS-B                run 19/100  MSE=1.710e-12  nit=15  nfev=57   t=10.78s
L-BFGS-B                run 20/100  MSE=5.000e-13  nit=12  nfev=63   t=12.01s
L-BFGS-B                run 21/100  MSE=5.719e-13  nit=13  nfev=54   t=9.99s
L-BFGS-B                run 22/100  MSE=7.150e-13  nit=12  nfev=51   t=9.53s
L-BFGS-B                run 23/100  MSE=1.677e-13  nit=9   nfev=48   t=9.92s
L-BFGS-B                run 24/100  MSE=9.660e-13  nit=9   nfev=45   t=9.18s
L-BFGS-B                run 25/100  MSE=4.119e-13  nit=15  nfev=51   t=10.38s
L-BFGS-B                run 26/100  MSE=4.992e-13  nit=15  nfev=57   t=10.97s
L-BFGS-B                run 27/100  MSE=9.095e-14  nit=11  nfev=48   t=9.05s
L-BFGS-B                run 28/100  MSE=9.397e-13  nit=8   nfev=51   t=9.52s
L-BFGS-B                run 29/100  MSE=2.368e-12  nit=16  nfev=72   t=13.41s
L-BFGS-B                run 30/100  MSE=5.674e-13  nit=14  nfev=60   t=11.34s
L-BFGS-B                run 31/100  MSE=3.898e-11  nit=11  nfev=42   t=8.03s
L-BFGS-B                run 32/100  MSE=1.424e-13  nit=10  nfev=42   t=8.06s
L-BFGS-B                run 33/100  MSE=7.927e-12  nit=13  nfev=48   t=9.08s
L-BFGS-B                run 34/100  MSE=1.493e-13  nit=10  nfev=51   t=9.96s
L-BFGS-B                run 35/100  MSE=4.371e-13  nit=14  nfev=60   t=11.67s
L-BFGS-B                run 36/100  MSE=1.188e-12  nit=13  nfev=45   t=8.57s
L-BFGS-B                run 37/100  MSE=5.074e-13  nit=11  nfev=39   t=7.46s
L-BFGS-B                run 38/100  MSE=6.124e-13  nit=13  nfev=57   t=10.68s
L-BFGS-B                run 39/100  MSE=3.511e-13  nit=12  nfev=66   t=12.37s
L-BFGS-B                run 40/100  MSE=6.622e-13  nit=15  nfev=51   t=9.51s
L-BFGS-B                run 41/100  MSE=5.598e-13  nit=13  nfev=51   t=9.85s
L-BFGS-B                run 42/100  MSE=5.057e-02  nit=8   nfev=93   t=17.51s
L-BFGS-B                run 43/100  MSE=4.252e-13  nit=8   nfev=39   t=7.53s
L-BFGS-B                run 44/100  MSE=4.595e-13  nit=14  nfev=48   t=9.47s
L-BFGS-B                run 45/100  MSE=5.116e-12  nit=13  nfev=54   t=10.24s
L-BFGS-B                run 46/100  MSE=1.071e-13  nit=11  nfev=48   t=9.30s
L-BFGS-B                run 47/100  MSE=1.577e-14  nit=9   nfev=45   t=8.44s
L-BFGS-B                run 48/100  MSE=4.339e-13  nit=8   nfev=51   t=9.46s
L-BFGS-B                run 49/100  MSE=4.653e-12  nit=10  nfev=54   t=10.38s
L-BFGS-B                run 50/100  MSE=7.708e-13  nit=10  nfev=54   t=10.13s
L-BFGS-B                run 51/100  MSE=1.342e-11  nit=10  nfev=48   t=9.20s
L-BFGS-B                run 52/100  MSE=9.331e-14  nit=12  nfev=54   t=10.14s
```

```
L-BFGS-B              run 53/100  MSE=2.774e-13  nit=10  nfev=51   t=9.77s
L-BFGS-B              run 54/100  MSE=3.798e-13  nit=15  nfev=72   t=13.47s
L-BFGS-B              run 55/100  MSE=1.693e-12  nit=9   nfev=57   t=10.77s
L-BFGS-B              run 56/100  MSE=3.244e-13  nit=8   nfev=36   t=6.71s
L-BFGS-B              run 57/100  MSE=3.049e-13  nit=16  nfev=54   t=10.07s
L-BFGS-B              run 58/100  MSE=8.555e-13  nit=8   nfev=48   t=8.97s
L-BFGS-B              run 59/100  MSE=1.086e-12  nit=12  nfev=51   t=10.02s
L-BFGS-B              run 60/100  MSE=5.599e-13  nit=16  nfev=69   t=13.32s
L-BFGS-B              run 61/100  MSE=2.003e-13  nit=11  nfev=42   t=7.97s
L-BFGS-B              run 62/100  MSE=5.677e-13  nit=12  nfev=48   t=9.38s
L-BFGS-B              run 63/100  MSE=5.368e-13  nit=14  nfev=51   t=9.91s
L-BFGS-B              run 64/100  MSE=3.796e-13  nit=12  nfev=42   t=7.84s
L-BFGS-B              run 65/100  MSE=9.248e-11  nit=12  nfev=60   t=11.24s
L-BFGS-B              run 66/100  MSE=9.629e-13  nit=14  nfev=54   t=10.21s
L-BFGS-B              run 67/100  MSE=9.212e-14  nit=10  nfev=60   t=11.41s
L-BFGS-B              run 68/100  MSE=2.176e-13  nit=9   nfev=42   t=8.08s
L-BFGS-B              run 69/100  MSE=6.106e-13  nit=11  nfev=54   t=10.23s
L-BFGS-B              run 70/100  MSE=6.328e-13  nit=11  nfev=48   t=9.11s
L-BFGS-B              run 71/100  MSE=1.228e-12  nit=15  nfev=57   t=11.03s
L-BFGS-B              run 72/100  MSE=3.443e-13  nit=12  nfev=51   t=9.51s
L-BFGS-B              run 73/100  MSE=3.655e-13  nit=12  nfev=42   t=7.88s
L-BFGS-B              run 74/100  MSE=5.004e-14  nit=6   nfev=45   t=8.66s
L-BFGS-B              run 75/100  MSE=1.313e-11  nit=15  nfev=54   t=10.16s
L-BFGS-B              run 76/100  MSE=1.435e-13  nit=10  nfev=45   t=8.47s
L-BFGS-B              run 77/100  MSE=4.896e-13  nit=9   nfev=39   t=7.50s
L-BFGS-B              run 78/100  MSE=5.269e-13  nit=18  nfev=81   t=15.29s
L-BFGS-B              run 79/100  MSE=1.520e-12  nit=11  nfev=51   t=9.91s
L-BFGS-B              run 80/100  MSE=1.734e-12  nit=14  nfev=57   t=11.07s
L-BFGS-B              run 81/100  MSE=2.168e-13  nit=14  nfev=54   t=10.08s
L-BFGS-B              run 82/100  MSE=4.009e-13  nit=8   nfev=51   t=9.54s
L-BFGS-B              run 83/100  MSE=4.565e-13  nit=12  nfev=45   t=8.46s
L-BFGS-B              run 84/100  MSE=4.319e-13  nit=8   nfev=51   t=9.93s
L-BFGS-B              run 85/100  MSE=1.402e-12  nit=13  nfev=72   t=13.74s
L-BFGS-B              run 86/100  MSE=8.495e-14  nit=14  nfev=48   t=9.20s
L-BFGS-B              run 87/100  MSE=1.415e-12  nit=9   nfev=39   t=7.16s
L-BFGS-B              run 88/100  MSE=4.573e-13  nit=8   nfev=36   t=6.85s
L-BFGS-B              run 89/100  MSE=7.976e-14  nit=9   nfev=75   t=14.36s
L-BFGS-B              run 90/100  MSE=6.427e-13  nit=12  nfev=45   t=8.26s
L-BFGS-B              run 91/100  MSE=8.707e-13  nit=12  nfev=42   t=7.79s
L-BFGS-B              run 92/100  MSE=3.251e-12  nit=6   nfev=39   t=7.52s
L-BFGS-B              run 93/100  MSE=1.352e-13  nit=14  nfev=51   t=9.47s
L-BFGS-B              run 94/100  MSE=6.276e-13  nit=13  nfev=63   t=11.86s
L-BFGS-B              run 95/100  MSE=6.007e-13  nit=18  nfev=72   t=13.59s
L-BFGS-B              run 96/100  MSE=3.119e-12  nit=10  nfev=42   t=7.76s
L-BFGS-B              run 97/100  MSE=2.037e-13  nit=14  nfev=54   t=10.17s
L-BFGS-B              run 98/100  MSE=1.197e-13  nit=10  nfev=42   t=8.12s
L-BFGS-B              run 99/100  MSE=1.496e-12  nit=14  nfev=54   t=10.13s
L-BFGS-B              run 100/100  MSE=2.564e-13  nit=11  nfev=48   t=9.04s
```

0.4.3 Scenario A — SLSQP $+\nabla f$ (100 runs)

```
SLSQP   run 1/100   MSE=4.087e-07  nit=9   nfev=29   t=5.45s
SLSQP   run 2/100   MSE=3.040e-07  nit=11  nfev=36   t=7.03s
SLSQP   run 3/100   MSE=2.472e-08  nit=12  nfev=39   t=7.26s
SLSQP   run 4/100   MSE=2.861e-08  nit=15  nfev=51   t=9.54s
SLSQP   run 5/100   MSE=6.313e-08  nit=10  nfev=33   t=6.49s
SLSQP   run 6/100   MSE=1.924e-07  nit=8   nfev=26   t=4.82s
SLSQP   run 7/100   MSE=5.724e-10  nit=11  nfev=34   t=6.51s
SLSQP   run 8/100   MSE=8.474e-09  nit=13  nfev=42   t=7.96s
SLSQP   run 9/100   MSE=6.403e-10  nit=7   nfev=23   t=4.21s
SLSQP   run 10/100  MSE=3.895e-09  nit=13  nfev=42   t=7.96s
SLSQP   run 11/100  MSE=1.945e-07  nit=5   nfev=17   t=3.33s
SLSQP   run 12/100  MSE=8.234e-10  nit=9   nfev=29   t=5.31s
SLSQP   run 13/100  MSE=1.322e-09  nit=21  nfev=70   t=12.97s
SLSQP   run 14/100  MSE=8.339e-10  nit=14  nfev=47   t=9.23s
SLSQP   run 15/100  MSE=3.295e-07  nit=8   nfev=27   t=4.99s
SLSQP   run 16/100  MSE=5.020e-08  nit=12  nfev=39   t=7.33s
SLSQP   run 17/100  MSE=8.971e-08  nit=10  nfev=33   t=6.04s
SLSQP   run 18/100  MSE=1.957e-07  nit=12  nfev=39   t=7.32s
SLSQP   run 19/100  MSE=7.554e-09  nit=14  nfev=46   t=8.84s
SLSQP   run 20/100  MSE=2.575e-09  nit=14  nfev=46   t=8.86s
SLSQP   run 21/100  MSE=3.746e-11  nit=11  nfev=35   t=6.68s
SLSQP   run 22/100  MSE=1.962e-07  nit=11  nfev=38   t=7.36s
SLSQP   run 23/100  MSE=3.386e-07  nit=14  nfev=46   t=8.67s
SLSQP   run 24/100  MSE=1.129e-08  nit=13  nfev=41   t=7.49s
SLSQP   run 25/100  MSE=1.824e-08  nit=12  nfev=42   t=7.77s
SLSQP   run 26/100  MSE=1.767e-07  nit=11  nfev=35   t=6.66s
SLSQP   run 27/100  MSE=1.374e-07  nit=14  nfev=47   t=8.89s
SLSQP   run 28/100  MSE=3.132e-08  nit=14  nfev=44   t=8.07s
SLSQP   run 29/100  MSE=1.765e-10  nit=9   nfev=31   t=5.80s
SLSQP   run 30/100  MSE=1.515e-07  nit=12  nfev=40   t=7.65s
SLSQP   run 31/100  MSE=1.414e-07  nit=8   nfev=27   t=5.17s
SLSQP   run 32/100  MSE=3.511e-08  nit=11  nfev=36   t=6.71s
SLSQP   run 33/100  MSE=1.742e-07  nit=15  nfev=50   t=9.36s
SLSQP   run 34/100  MSE=5.715e-08  nit=18  nfev=61   t=11.79s
SLSQP   run 35/100  MSE=2.897e-08  nit=12  nfev=40   t=7.41s
SLSQP   run 36/100  MSE=7.476e-08  nit=17  nfev=58   t=10.83s
SLSQP   run 37/100  MSE=1.038e-07  nit=13  nfev=44   t=8.43s
SLSQP   run 38/100  MSE=1.409e-08  nit=12  nfev=41   t=7.60s
SLSQP   run 39/100  MSE=2.297e-10  nit=9   nfev=30   t=5.60s
SLSQP   run 40/100  MSE=6.751e-09  nit=13  nfev=43   t=8.06s
SLSQP   run 41/100  MSE=1.622e-08  nit=11  nfev=36   t=6.81s
SLSQP   run 42/100  MSE=2.935e-07  nit=8   nfev=26   t=5.03s
SLSQP   run 43/100  MSE=3.955e-07  nit=6   nfev=21   t=3.77s
SLSQP   run 44/100  MSE=2.886e-08  nit=13  nfev=45   t=8.40s
SLSQP   run 45/100  MSE=2.288e-09  nit=8   nfev=27   t=5.56s
SLSQP   run 46/100  MSE=1.559e-08  nit=12  nfev=42   t=8.00s
```

```
SLSQP                run 47/100  MSE=2.110e-07  nit=11  nfev=36   t=6.82s
SLSQP                run 48/100  MSE=7.598e-10  nit=12  nfev=40   t=7.47s
SLSQP                run 49/100  MSE=1.459e-07  nit=8   nfev=26   t=4.84s
SLSQP                run 50/100  MSE=8.711e-09  nit=14  nfev=47   t=8.79s
SLSQP                run 51/100  MSE=3.009e-09  nit=12  nfev=40   t=7.72s
SLSQP                run 52/100  MSE=1.289e-07  nit=15  nfev=49   t=8.90s
SLSQP                run 53/100  MSE=4.431e-07  nit=14  nfev=45   t=8.39s
SLSQP                run 54/100  MSE=3.933e-07  nit=10  nfev=31   t=5.69s
SLSQP                run 55/100  MSE=3.038e-08  nit=8   nfev=26   t=4.98s
SLSQP                run 56/100  MSE=1.113e-08  nit=8   nfev=29   t=5.50s
SLSQP                run 57/100  MSE=3.674e-08  nit=13  nfev=43   t=7.93s
SLSQP                run 58/100  MSE=5.458e-08  nit=13  nfev=42   t=7.73s
SLSQP                run 59/100  MSE=6.014e-07  nit=17  nfev=57   t=10.85s
SLSQP                run 60/100  MSE=3.662e-07  nit=9   nfev=31   t=5.77s
SLSQP                run 61/100  MSE=5.193e-09  nit=7   nfev=23   t=4.97s
SLSQP                run 62/100  MSE=3.318e-07  nit=8   nfev=26   t=4.90s
SLSQP                run 63/100  MSE=9.253e-09  nit=13  nfev=43   t=8.10s
SLSQP                run 64/100  MSE=2.512e-07  nit=17  nfev=57   t=13.73s
SLSQP                run 65/100  MSE=3.100e-08  nit=12  nfev=40   t=9.73s
SLSQP                run 66/100  MSE=1.943e-08  nit=11  nfev=35   t=6.49s
SLSQP                run 67/100  MSE=2.990e-07  nit=8   nfev=27   t=5.54s
SLSQP                run 68/100  MSE=5.841e-09  nit=7   nfev=24   t=4.65s
SLSQP                run 69/100  MSE=1.457e-07  nit=12  nfev=39   t=7.24s
SLSQP                run 70/100  MSE=3.188e-09  nit=16  nfev=54   t=10.17s
SLSQP                run 71/100  MSE=2.187e-07  nit=15  nfev=49   t=9.22s
SLSQP                run 72/100  MSE=8.832e-09  nit=12  nfev=38   t=7.07s
SLSQP                run 73/100  MSE=4.251e-08  nit=7   nfev=23   t=4.29s
SLSQP                run 74/100  MSE=1.570e-07  nit=12  nfev=38   t=7.13s
SLSQP                run 75/100  MSE=1.328e-03  nit=9   nfev=31   t=6.04s
SLSQP                run 76/100  MSE=8.750e-09  nit=11  nfev=38   t=7.15s
SLSQP                run 77/100  MSE=4.898e-09  nit=8   nfev=28   t=5.29s
SLSQP                run 78/100  MSE=3.587e-08  nit=10  nfev=33   t=6.18s
SLSQP                run 79/100  MSE=5.534e-09  nit=17  nfev=57   t=11.02s
SLSQP                run 80/100  MSE=1.439e-07  nit=12  nfev=39   t=7.26s
SLSQP                run 81/100  MSE=3.450e-08  nit=12  nfev=38   t=6.99s
SLSQP                run 82/100  MSE=2.905e-07  nit=8   nfev=29   t=5.49s
SLSQP                run 83/100  MSE=2.106e-07  nit=9   nfev=31   t=5.88s
SLSQP                run 84/100  MSE=5.300e-08  nit=8   nfev=27   t=5.15s
SLSQP                run 85/100  MSE=9.253e-10  nit=9   nfev=31   t=5.70s
SLSQP                run 86/100  MSE=1.265e-10  nit=11  nfev=36   t=6.71s
SLSQP                run 87/100  MSE=9.720e-08  nit=11  nfev=37   t=7.12s
SLSQP                run 88/100  MSE=4.640e-07  nit=7   nfev=24   t=4.48s
SLSQP                run 89/100  MSE=9.714e-08  nit=9   nfev=29   t=5.69s
SLSQP                run 90/100  MSE=1.552e-07  nit=10  nfev=34   t=6.43s
SLSQP                run 91/100  MSE=2.987e-08  nit=8   nfev=26   t=4.85s
SLSQP                run 92/100  MSE=3.436e-08  nit=6   nfev=22   t=4.07s
SLSQP                run 93/100  MSE=1.490e-07  nit=11  nfev=35   t=6.64s
SLSQP                run 94/100  MSE=5.068e-08  nit=13  nfev=42   t=8.09s
```

```
SLSQP                run 95/100  MSE=9.628e-08  nit=16  nfev=54   t=10.10s
SLSQP                run 96/100  MSE=1.968e-08  nit=11  nfev=36   t=6.78s
SLSQP                run 97/100  MSE=2.979e-07  nit=10  nfev=33   t=6.27s
SLSQP                run 98/100  MSE=6.954e-09  nit=12  nfev=43   t=8.23s
SLSQP                run 99/100  MSE=7.888e-09  nit=12  nfev=38   t=7.48s
SLSQP                run 100/100  MSE=7.412e-08  nit=12  nfev=41   t=7.65s
```

0.4.4 Scenario A — COBYLA (100 runs)

```
COBYLA               run 1/100  MSE=7.439e-05  nit=-1  nfev=141  t=26.95s
COBYLA               run 2/100  MSE=6.869e-05  nit=-1  nfev=123  t=23.43s
COBYLA               run 3/100  MSE=3.328e-05  nit=-1  nfev=544  t=104.04s
COBYLA               run 4/100  MSE=7.737e-05  nit=-1  nfev=438  t=83.42s
COBYLA               run 5/100  MSE=1.308e-05  nit=-1  nfev=105  t=19.97s
COBYLA               run 6/100  MSE=6.556e-05  nit=-1  nfev=567  t=109.49s
COBYLA               run 7/100  MSE=6.218e-05  nit=-1  nfev=134  t=20.95s
COBYLA               run 8/100  MSE=7.302e-05  nit=-1  nfev=85   t=13.33s
COBYLA               run 9/100  MSE=5.100e-05  nit=-1  nfev=430  t=66.54s
COBYLA               run 10/100  MSE=3.558e-05  nit=-1  nfev=156  t=24.05s
COBYLA               run 11/100  MSE=5.100e-05  nit=-1  nfev=430  t=77.24s
COBYLA               run 12/100  MSE=4.381e-05  nit=-1  nfev=578  t=116.09s
COBYLA               run 13/100  MSE=4.126e-05  nit=-1  nfev=99   t=17.44s
COBYLA               run 14/100  MSE=6.332e-05  nit=-1  nfev=579  t=108.09s
COBYLA               run 15/100  MSE=5.301e-05  nit=-1  nfev=210  t=38.82s
COBYLA               run 16/100  MSE=2.560e-05  nit=-1  nfev=69   t=12.42s
COBYLA               run 17/100  MSE=5.100e-05  nit=-1  nfev=430  t=78.47s
COBYLA               run 18/100  MSE=3.946e-05  nit=-1  nfev=123  t=20.73s
COBYLA               run 19/100  MSE=1.052e-04  nit=-1  nfev=167  t=28.92s
COBYLA               run 20/100  MSE=9.845e-08  nit=-1  nfev=31   t=5.66s
COBYLA               run 21/100  MSE=3.314e-05  nit=-1  nfev=123  t=22.51s
COBYLA               run 22/100  MSE=7.347e-07  nit=-1  nfev=69   t=13.05s
COBYLA               run 23/100  MSE=7.261e-05  nit=-1  nfev=573  t=103.48s
COBYLA               run 24/100  MSE=4.706e-05  nit=-1  nfev=131  t=23.72s
COBYLA               run 25/100  MSE=1.125e-04  nit=-1  nfev=127  t=22.91s
COBYLA               run 26/100  MSE=4.489e-06  nit=-1  nfev=80   t=15.08s
COBYLA               run 27/100  MSE=1.284e-04  nit=-1  nfev=102  t=19.68s
COBYLA               run 28/100  MSE=1.752e-05  nit=-1  nfev=167  t=27.87s
COBYLA               run 29/100  MSE=5.419e-05  nit=-1  nfev=492  t=76.15s
COBYLA               run 30/100  MSE=1.328e-05  nit=-1  nfev=128  t=19.80s
COBYLA               run 31/100  MSE=1.195e-05  nit=-1  nfev=40   t=6.36s
COBYLA               run 32/100  MSE=1.280e-04  nit=-1  nfev=104  t=16.53s
COBYLA               run 33/100  MSE=4.869e-05  nit=-1  nfev=128  t=19.48s
COBYLA               run 34/100  MSE=7.513e-05  nit=-1  nfev=131  t=20.10s
COBYLA               run 35/100  MSE=8.017e-05  nit=-1  nfev=108  t=16.78s
COBYLA               run 36/100  MSE=4.913e-05  nit=-1  nfev=495  t=76.32s
COBYLA               run 37/100  MSE=5.100e-05  nit=-1  nfev=430  t=67.06s
COBYLA               run 38/100  MSE=3.270e-05  nit=-1  nfev=138  t=21.20s
COBYLA               run 39/100  MSE=5.100e-05  nit=-1  nfev=430  t=66.53s
```

```
COBYLA                run 40/100  MSE=6.283e-06  nit=-1  nfev=236  t=37.04s
COBYLA                run 41/100  MSE=2.024e-05  nit=-1  nfev=278  t=42.86s
COBYLA                run 42/100  MSE=8.518e-02  nit=-1  nfev=41   t=6.43s
COBYLA                run 43/100  MSE=1.108e-04  nit=-1  nfev=532  t=82.84s
COBYLA                run 44/100  MSE=3.963e-07  nit=-1  nfev=161  t=25.04s
COBYLA                run 45/100  MSE=6.442e-05  nit=-1  nfev=564  t=87.11s
COBYLA                run 46/100  MSE=3.170e-05  nit=-1  nfev=150  t=23.10s
COBYLA                run 47/100  MSE=8.100e-05  nit=-1  nfev=112  t=17.81s
COBYLA                run 48/100  MSE=2.299e-06  nit=-1  nfev=85   t=13.05s
COBYLA                run 49/100  MSE=9.395e-05  nit=-1  nfev=438  t=67.49s
COBYLA                run 50/100  MSE=3.389e-05  nit=-1  nfev=162  t=25.16s
COBYLA                run 51/100  MSE=3.328e-05  nit=-1  nfev=544  t=84.98s
COBYLA                run 52/100  MSE=1.141e-06  nit=-1  nfev=33   t=5.22s
COBYLA                run 53/100  MSE=1.632e-05  nit=-1  nfev=134  t=20.74s
COBYLA                run 54/100  MSE=2.588e-05  nit=-1  nfev=152  t=23.47s
COBYLA                run 55/100  MSE=8.360e-05  nit=-1  nfev=97   t=15.00s
COBYLA                run 56/100  MSE=3.129e-05  nit=-1  nfev=450  t=70.21s
COBYLA                run 57/100  MSE=2.112e-07  nit=-1  nfev=112  t=17.45s
COBYLA                run 58/100  MSE=7.772e-05  nit=-1  nfev=592  t=91.11s
COBYLA                run 59/100  MSE=7.456e-05  nit=-1  nfev=121  t=18.72s
COBYLA                run 60/100  MSE=2.372e-05  nit=-1  nfev=186  t=28.56s
COBYLA                run 61/100  MSE=7.589e-05  nit=-1  nfev=576  t=89.15s
COBYLA                run 62/100  MSE=4.820e-05  nit=-1  nfev=190  t=29.41s
COBYLA                run 63/100  MSE=1.306e-06  nit=-1  nfev=126  t=19.67s
COBYLA                run 64/100  MSE=2.508e-05  nit=-1  nfev=120  t=18.67s
COBYLA                run 65/100  MSE=3.217e-05  nit=-1  nfev=146  t=22.70s
COBYLA                run 66/100  MSE=1.438e-07  nit=-1  nfev=98   t=15.09s
COBYLA                run 67/100  MSE=3.328e-05  nit=-1  nfev=543  t=86.10s
COBYLA                run 68/100  MSE=5.416e-06  nit=-1  nfev=529  t=81.50s
COBYLA                run 69/100  MSE=1.446e-05  nit=-1  nfev=108  t=16.97s
COBYLA                run 70/100  MSE=3.458e-05  nit=-1  nfev=43   t=6.47s
COBYLA                run 71/100  MSE=1.903e-05  nit=-1  nfev=152  t=23.59s
COBYLA                run 72/100  MSE=5.542e-05  nit=-1  nfev=148  t=23.95s
COBYLA                run 73/100  MSE=1.004e-05  nit=-1  nfev=534  t=83.22s
COBYLA                run 74/100  MSE=5.127e-05  nit=-1  nfev=134  t=20.81s
COBYLA                run 75/100  MSE=6.063e-06  nit=-1  nfev=99   t=15.41s
COBYLA                run 76/100  MSE=5.103e-05  nit=-1  nfev=104  t=15.69s
COBYLA                run 77/100  MSE=4.646e-05  nit=-1  nfev=191  t=29.63s
COBYLA                run 78/100  MSE=4.463e-05  nit=-1  nfev=104  t=16.22s
COBYLA                run 79/100  MSE=1.167e-04  nit=-1  nfev=473  t=73.04s
COBYLA                run 80/100  MSE=4.481e-05  nit=-1  nfev=141  t=21.77s
COBYLA                run 81/100  MSE=4.005e-05  nit=-1  nfev=321  t=49.49s
COBYLA                run 82/100  MSE=8.533e-02  nit=-1  nfev=31   t=4.79s
COBYLA                run 83/100  MSE=1.701e-05  nit=-1  nfev=145  t=22.42s
COBYLA                run 84/100  MSE=8.267e-05  nit=-1  nfev=95   t=14.61s
COBYLA                run 85/100  MSE=7.548e-05  nit=-1  nfev=75   t=11.45s
COBYLA                run 86/100  MSE=5.100e-05  nit=-1  nfev=430  t=67.38s
COBYLA                run 87/100  MSE=3.647e-07  nit=-1  nfev=95   t=14.59s
```

```
COBYLA                run 88/100  MSE=1.004e-05  nit=-1  nfev=534  t=82.69s
COBYLA                run 89/100  MSE=7.075e-04  nit=-1  nfev=31   t=4.86s
COBYLA                run 90/100  MSE=6.831e-05  nit=-1  nfev=200  t=31.05s
COBYLA                run 91/100  MSE=1.004e-05  nit=-1  nfev=534  t=82.17s
COBYLA                run 92/100  MSE=7.166e-06  nit=-1  nfev=80   t=12.15s
COBYLA                run 93/100  MSE=6.141e-05  nit=-1  nfev=232  t=35.49s
COBYLA                run 94/100  MSE=1.222e-04  nit=-1  nfev=138  t=21.71s
COBYLA                run 95/100  MSE=6.405e-05  nit=-1  nfev=119  t=18.29s
COBYLA                run 96/100  MSE=1.773e-07  nit=-1  nfev=35   t=9.07s
COBYLA                run 97/100  MSE=3.077e-08  nit=-1  nfev=126  t=20.70s
COBYLA                run 98/100  MSE=6.642e-05  nit=-1  nfev=123  t=18.93s
COBYLA                run 99/100  MSE=4.858e-07  nit=-1  nfev=95   t=15.92s
COBYLA                run 100/100  MSE=3.220e-05  nit=-1  nfev=561  t=85.82s
```

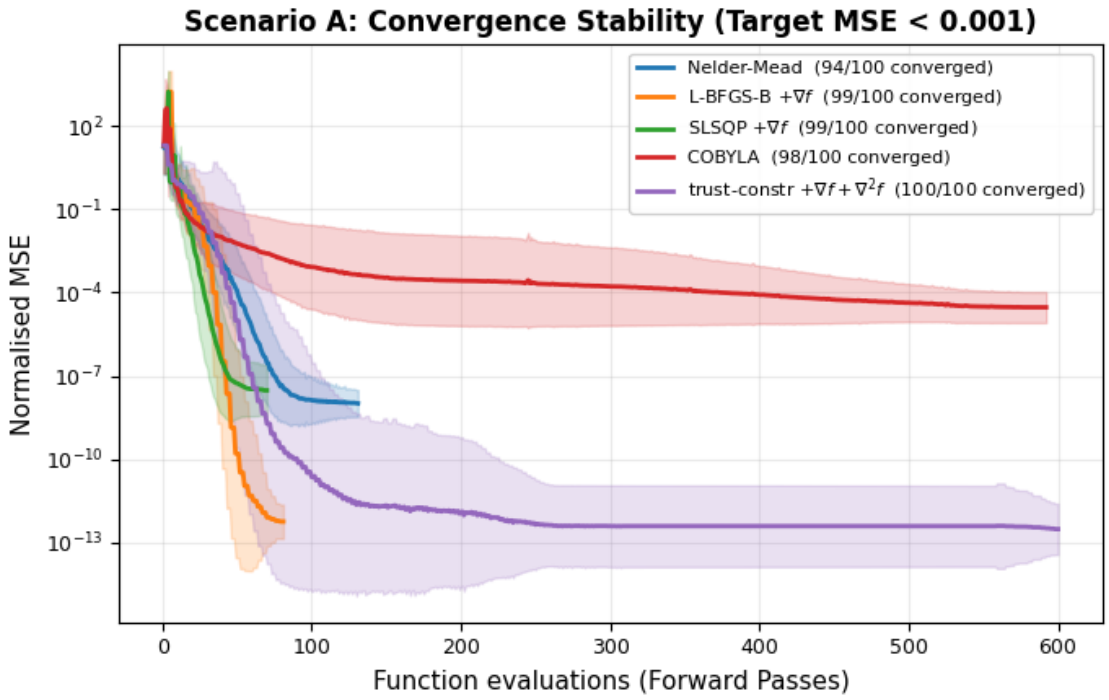
0.4.5 Scenario A — trust-constr $+\nabla f + \nabla^2 f$ (100 runs)

```
trust-constr          run 1/100  MSE=9.351e-15  nit=22  nfev=51   t=7.92s
trust-constr          run 2/100  MSE=6.277e-14  nit=21  nfev=42   t=6.53s
trust-constr          run 3/100  MSE=6.251e-14  nit=71  nfev=186  t=28.49s
trust-constr          run 4/100  MSE=7.464e-13  nit=39  nfev=96   t=14.72s
trust-constr          run 5/100  MSE=2.253e-12  nit=32  nfev=84   t=12.84s
trust-constr          run 6/100  MSE=6.270e-14  nit=25  nfev=54   t=8.27s
trust-constr          run 7/100  MSE=6.243e-14  nit=80  nfev=222  t=33.77s
trust-constr          run 8/100  MSE=1.790e-12  nit=31  nfev=72   t=11.03s
trust-constr          run 9/100  MSE=1.768e-12  nit=27  nfev=60   t=9.30s
trust-constr          run 10/100  MSE=6.195e-14  nit=43  nfev=108  t=17.21s
trust-constr          run 11/100  MSE=1.768e-12  nit=25  nfev=54   t=8.20s
trust-constr          run 12/100  MSE=1.768e-12  nit=25  nfev=54   t=8.33s
trust-constr          run 13/100  MSE=1.789e-12  nit=30  nfev=69   t=10.57s
trust-constr          run 14/100  MSE=1.387e-11  nit=26  nfev=60   t=9.03s
trust-constr          run 15/100  MSE=1.790e-12  nit=28  nfev=63   t=9.68s
trust-constr          run 16/100  MSE=7.464e-13  nit=34  nfev=81   t=12.69s
trust-constr          run 17/100  MSE=6.328e-14  nit=75  nfev=213  t=32.57s
trust-constr          run 18/100  MSE=6.268e-14  nit=28  nfev=66   t=9.93s
trust-constr          run 19/100  MSE=9.727e-15  nit=19  nfev=39   t=6.13s
trust-constr          run 20/100  MSE=9.950e-16  nit=82  nfev=222  t=34.11s
trust-constr          run 21/100  MSE=6.275e-14  nit=45  nfev=120  t=18.36s
trust-constr          run 22/100  MSE=1.790e-12  nit=43  nfev=108  t=16.66s
trust-constr          run 23/100  MSE=4.676e-15  nit=76  nfev=204  t=31.10s
trust-constr          run 24/100  MSE=6.269e-14  nit=26  nfev=57   t=8.62s
trust-constr          run 25/100  MSE=1.768e-12  nit=205 nfev=600  t=91.56s
trust-constr          run 26/100  MSE=9.313e-15  nit=20  nfev=42   t=6.56s
trust-constr          run 27/100  MSE=7.465e-13  nit=96  nfev=267  t=40.90s
trust-constr          run 28/100  MSE=6.264e-14  nit=27  nfev=60   t=9.32s
trust-constr          run 29/100  MSE=1.333e-14  nit=80  nfev=216  t=33.00s
trust-constr          run 30/100  MSE=1.790e-12  nit=55  nfev=147  t=22.72s
trust-constr          run 31/100  MSE=6.279e-14  nit=22  nfev=45   t=7.00s
trust-constr          run 32/100  MSE=1.346e-15  nit=81  nfev=222  t=34.62s
```

```
trust-constr           run 33/100  MSE=1.790e-12  nit=32  nfev=75   t=11.48s
trust-constr           run 34/100  MSE=1.790e-12  nit=30  nfev=69   t=10.56s
trust-constr           run 35/100  MSE=4.074e-17  nit=86  nfev=237  t=36.02s
trust-constr           run 36/100  MSE=7.466e-13  nit=28  nfev=63   t=9.62s
trust-constr           run 37/100  MSE=6.916e-14  nit=33  nfev=78   t=12.15s
trust-constr           run 38/100  MSE=9.578e-16  nit=95  nfev=270  t=42.35s
trust-constr           run 39/100  MSE=9.358e-15  nit=42  nfev=111  t=17.67s
trust-constr           run 40/100  MSE=1.790e-12  nit=25  nfev=54   t=8.60s
trust-constr           run 41/100  MSE=7.464e-13  nit=35  nfev=90   t=15.93s
trust-constr           run 42/100  MSE=1.790e-12  nit=30  nfev=72   t=11.30s
trust-constr           run 43/100  MSE=1.768e-12  nit=25  nfev=54   t=8.21s
trust-constr           run 44/100  MSE=1.768e-12  nit=39  nfev=96   t=14.77s
trust-constr           run 45/100  MSE=1.790e-12  nit=33  nfev=81   t=12.34s
trust-constr           run 46/100  MSE=1.790e-12  nit=33  nfev=81   t=12.36s
trust-constr           run 47/100  MSE=1.789e-12  nit=30  nfev=69   t=10.55s
trust-constr           run 48/100  MSE=1.790e-12  nit=30  nfev=69   t=10.63s
trust-constr           run 49/100  MSE=6.268e-14  nit=26  nfev=57   t=8.56s
trust-constr           run 50/100  MSE=1.386e-11  nit=38  nfev=102  t=15.64s
trust-constr           run 51/100  MSE=7.468e-13  nit=34  nfev=81   t=12.31s
trust-constr           run 52/100  MSE=1.306e-16  nit=86  nfev=237  t=36.71s
trust-constr           run 53/100  MSE=6.270e-14  nit=69  nfev=186  t=28.15s
trust-constr           run 54/100  MSE=6.268e-14  nit=49  nfev=129  t=19.73s
trust-constr           run 55/100  MSE=8.704e-16  nit=110 nfev=309  t=47.22s
trust-constr           run 56/100  MSE=1.912e-15  nit=78  nfev=210  t=33.01s
trust-constr           run 57/100  MSE=1.386e-11  nit=23  nfev=51   t=7.83s
trust-constr           run 58/100  MSE=6.276e-14  nit=21  nfev=42   t=6.39s
trust-constr           run 59/100  MSE=1.790e-12  nit=92  nfev=255  t=38.72s
trust-constr           run 60/100  MSE=9.532e-15  nit=31  nfev=78   t=12.01s
trust-constr           run 61/100  MSE=1.768e-12  nit=27  nfev=60   t=9.24s
trust-constr           run 62/100  MSE=1.790e-12  nit=36  nfev=90   t=13.64s
trust-constr           run 63/100  MSE=1.790e-12  nit=25  nfev=54   t=8.49s
trust-constr           run 64/100  MSE=6.305e-14  nit=65  nfev=171  t=26.43s
trust-constr           run 65/100  MSE=2.409e-16  nit=102 nfev=285  t=43.17s
trust-constr           run 66/100  MSE=2.251e-12  nit=25  nfev=54   t=8.13s
trust-constr           run 67/100  MSE=1.789e-12  nit=35  nfev=87   t=13.10s
trust-constr           run 68/100  MSE=6.316e-14  nit=31  nfev=72   t=11.14s
trust-constr           run 69/100  MSE=1.789e-12  nit=48  nfev=126  t=19.16s
trust-constr           run 70/100  MSE=7.467e-13  nit=32  nfev=72   t=11.02s
trust-constr           run 71/100  MSE=9.240e-16  nit=87  nfev=237  t=35.75s
trust-constr           run 72/100  MSE=7.464e-13  nit=55  nfev=144  t=22.45s
trust-constr           run 73/100  MSE=1.768e-12  nit=29  nfev=66   t=10.27s
trust-constr           run 74/100  MSE=1.128e-15  nit=103 nfev=291  t=44.11s
trust-constr           run 75/100  MSE=1.789e-12  nit=28  nfev=66   t=10.00s
trust-constr           run 76/100  MSE=9.505e-15  nit=24  nfev=57   t=8.78s
trust-constr           run 77/100  MSE=1.789e-12  nit=25  nfev=54   t=8.14s
trust-constr           run 78/100  MSE=5.919e-13  nit=36  nfev=84   t=12.63s
trust-constr           run 79/100  MSE=1.268e-15  nit=76  nfev=204  t=31.31s
trust-constr           run 80/100  MSE=6.270e-14  nit=39  nfev=102  t=15.45s
```

```
  trust-constr            run 81/100  MSE=2.251e-12  nit=28  nfev=63   t=9.55s
  trust-constr            run 82/100  MSE=9.216e-15  nit=31  nfev=75   t=11.24s
  trust-constr            run 83/100  MSE=2.251e-12  nit=30  nfev=69   t=10.70s
  trust-constr            run 84/100  MSE=9.629e-15  nit=21  nfev=45   t=6.82s
  trust-constr            run 85/100  MSE=7.463e-13  nit=31  nfev=72   t=10.90s
  trust-constr            run 86/100  MSE=2.205e-15  nit=90  nfev=246  t=39.06s
  trust-constr            run 87/100  MSE=1.790e-12  nit=33  nfev=81   t=12.84s
  trust-constr            run 88/100  MSE=1.768e-12  nit=28  nfev=63   t=10.18s
  trust-constr            run 89/100  MSE=6.287e-14  nit=84  nfev=231  t=36.70s
  trust-constr            run 90/100  MSE=2.073e-11  nit=25  nfev=60   t=9.42s
  trust-constr            run 91/100  MSE=1.768e-12  nit=30  nfev=69   t=10.91s
  trust-constr            run 92/100  MSE=1.168e-15  nit=79  nfev=213  t=32.59s
  trust-constr            run 93/100  MSE=2.251e-12  nit=28  nfev=63   t=9.52s
  trust-constr            run 94/100  MSE=6.269e-14  nit=39  nfev=99   t=15.56s
  trust-constr            run 95/100  MSE=1.789e-12  nit=25  nfev=54   t=8.13s
  trust-constr            run 96/100  MSE=3.056e-15  nit=78  nfev=213  t=33.24s
  trust-constr            run 97/100  MSE=2.056e-11  nit=27  nfev=63   t=10.16s
  trust-constr            run 98/100  MSE=7.465e-13  nit=33  nfev=78   t=12.07s
  trust-constr            run 99/100  MSE=1.790e-12  nit=30  nfev=72   t=11.03s
  trust-constr            run 100/100  MSE=1.790e-12  nit=27  nfev=66   t=9.99s

================================================================================
==================================
   SCENARIO A: Robustness (Tol < 0.001) & Discovered Physics State vs Reference
================================================================================
==================================
             method_tag             conv_rate nit_mean nfev_mean wall_time_mean
final_mse_mean I_1_ref    I_1    I_3_ref    I_3     alpha_x   alpha_y
                              COBYLA     98%    231.7     231.7      37.867 s
2.28e-05    0.8218 A 0.7494 A 1.0430 A 0.5834 A  5.49e-04  1.69e-03
               L-BFGS-B $+\nabla f$     99%     11.7      52.1       9.944 s
5.85e-13    0.8218 A 0.7494 A 1.0430 A 0.5834 A -2.58e-07 -4.67e-08
                         Nelder-Mead     94%     39.2      74.7      15.722 s
6.76e-09    0.8218 A 0.9470 A 1.0430 A 1.1846 A  1.48e-04 -4.81e-05
                     SLSQP $+\nabla f$     99%     11.3      37.4       7.124 s
3.04e-08    0.8218 A 0.7494 A 1.0430 A 0.5834 A  3.13e-07 -8.79e-06
trust-constr $+\nabla f+\nabla^2 f$    100%     45.5     116.6      17.956 s
1.81e-13    0.8218 A 0.7494 A 1.0430 A 0.5834 A -9.56e-08 -2.04e-07
================================================================================
==================================
```




```
   Scenario A: Best Optimised Quadrupole Currents per Method
Method                          Best MSE      I (A)      I2 (A)

  Nelder-Mead                5.2951e-10     0.9470     1.1846
  L-BFGS-B $+\nabla f$       1.5772e-14     0.7494     0.5834
  SLSQP $+\nabla f$          3.7457e-11     0.7494     0.5834
  COBYLA                     3.0770e-08     0.7494     0.5834
  trust-constr $+\nabla f+\nabla^2 f$   4.0738e-17     0.7494     0.5834

  Reference FELsim S1 currents: I=0.8218 A,  I2=1.0430 A

  Overall best across all methods/runs:
    Method: trust-constr $+\nabla f+\nabla^2 f$,  MSE=4.0738e-17
    I=0.74941 A,  I2=0.58336 A
time: 2h 28min 21s (started: 2026-06-07 21:50:09 -10:00)
```

[ ]:
```
# save the results
import pickle

with open(os.path.join(CURRENT_DIR, "../results/scenario_A_results_v3.pkl"),␣
 ↪"wb") as f:
    pickle.dump({
        "results_A": results_A,
        "df_A": df_A,
```

```
        "final_summary": final_summary
    }, f)
print("Results saved to scenario_A_results_v3.pkl")
```

```
     run        method scenario    method_tag     final_mse  nfev  nit  \
0      1   Nelder-Mead        A   Nelder-Mead  1.558643e-08    84   44
1      2   Nelder-Mead        A   Nelder-Mead  9.229138e-09    73   37
2      3   Nelder-Mead        A   Nelder-Mead  9.465581e-09   102   53
3      4   Nelder-Mead        A   Nelder-Mead  3.242282e-08    53   27
4      5   Nelder-Mead        A   Nelder-Mead  2.836254e-09    77   40
..   …           …      …           …            …   …  …
495   96  trust-constr        A  trust-constr  8.767633e-13    66   28
496   97  trust-constr        A  trust-constr  5.635462e-12    51   23
497   98  trust-constr        A  trust-constr  8.765611e-13    63   28
498   99  trust-constr        A  trust-constr  4.217449e-13    48   24
499  100  trust-constr        A  trust-constr  7.571811e-14    42   21

     wall_time  success       I_1       I_3       alpha_x       alpha_y  \
0       13.999     True  0.833853  1.119302 -1.291660e-04  1.757771e-04
1       13.887     True  0.833795  1.119254  6.180245e-05 -1.852240e-04
2       16.631     True  0.833804  1.119270 -9.330594e-05 -1.011195e-04
3        8.713     True  0.751151  0.580556  2.247880e-04  1.196494e-04
4       12.208     True  0.250209  0.240323 -1.104218e-04 -3.215809e-05
..         …      …       …         …            …            …
495     10.286     True  0.833826  1.119277 -1.287891e-06  1.534302e-07
496      7.926     True  0.833826  1.119277 -3.170306e-06  9.225020e-07
497      9.128     True  0.833826  1.119277 -1.286385e-06  1.570427e-07
498      7.383     True  0.833826  1.119278 -9.111392e-07  2.089892e-09
499      5.875     True  0.751184  0.580568  4.070499e-07  1.054134e-07

     is_converged
0            True
1            True
2            True
3            True
4            True
..            …
495          True
496          True
497          True
498          True
499          True

[500 rows x 14 columns]
Results saved to scenario_A_results_v3.pkl
time: 563 ms (started: 2026-06-06 12:53:26 -10:00)
```

[ ]:
```
from scipy.optimize import minimize, BFGS, SR1
import numpy as np

def f(x): return (x[0]-1)**2 + (x[1]-2)**2

r_none = minimize(f, [0,0], method='trust-constr', jac=None)
r_bfgs = minimize(f, [0,0], method='trust-constr', jac='2-point', hess=BFGS())

print("hess=None  :", np.round(r_none.x,6), "nit=", r_none.nit, "nfev=", r_none.
 ↪nfev)
print("hess=BFGS():", np.round(r_bfgs.x,6), "nit=", r_bfgs.nit, "nfev=", r_bfgs.
 ↪nfev)
```

```
hess=None  : [1. 2.] nit= 4 nfev= 12
hess=BFGS(): [1. 2.] nit= 4 nfev= 12
time: 203 ms (started: 2026-06-06 16:17:45 -10:00)
```

[ ]:
```
import importlib
import beamOptimizer as beamOptimizer_module
importlib.reload(beamOptimizer_module)
from beamOptimizer import beamOptimizer
```

```
time: 94 ms (started: 2026-06-06 17:13:22 -10:00)
```

[ ]:
```
import numpy as np, copy, pickle, os, time

bl = ExcelElements(EXCEL_PATH).create_beamline()[:A_BEAMLINE_LEN]
opti = beamOptimizer(bl, PARTICLES.copy())

SP = {"I":  {"bounds": CURRENT_BOUNDS, "start": 0.5},
      "I2": {"bounds": CURRENT_BOUNDS, "start": 0.5}}

def mse_at(i1, i2):
    try:
        v = opti.evaluate([i1, i2], A_VARS, SP, copy.deepcopy(A_OBJ))
        return v if np.isfinite(v) else np.nan
    except Exception:
        return np.nan

N_GRID = 60
lo, hi = 0.01, 1.5
I1_vals = np.linspace(lo, hi, N_GRID)
I2_vals = np.linspace(lo, hi, N_GRID)
MSE_grid = np.empty((N_GRID, N_GRID))

t0 = time.perf_counter()
for j, i2 in enumerate(I2_vals):
    for i, i1 in enumerate(I1_vals):
```

```
        MSE_grid[j, i] = mse_at(i1, i2)
    if (j+1) % 5 == 0:
        print(f"  row {j+1}/{N_GRID}   {time.perf_counter()-t0:.0f}s")

out_dir = os.path.join(CURRENT_DIR, "../results")
os.makedirs(out_dir, exist_ok=True)
with open(os.path.join(out_dir, "scenario_A_landscape.pkl"), "wb") as f:
    pickle.dump({"I1_vals": I1_vals, "I2_vals": I2_vals,
                 "MSE_grid": MSE_grid, "N_GRID": N_GRID, "bounds": (lo, hi)}, f)
print("  landscape saved")
```

```
  row 5/60   59s
  row 10/60   110s
  row 15/60   162s
  row 20/60   213s
  row 25/60   264s
  row 30/60   317s
  row 35/60   368s
  row 40/60   418s
  row 45/60   469s
  row 50/60   520s
  row 55/60   573s
  row 60/60   627s
  landscape saved
time: 10min 27s (started: 2026-06-06 17:13:25 -10:00)
```

[ ]:
```
import numpy as np, copy, pickle, os

N_STARTS = 5
rng = np.random.default_rng(2024)
STARTS = rng.uniform(0.01, 1.5, size=(N_STARTS, 2))
print("5 start points:\n", np.round(STARTS, 3))

TRAJ_METHODS = [
    ("Nelder-Mead",  None),
    ("L-BFGS-B",     "2-point"),
    ("SLSQP",        "2-point"),
    ("COBYLA",       None),
    ("trust-constr", "2-point"),
]

trajectories = {}
for method, jac in TRAJ_METHODS:
    method_trajs = []
    opts = METHOD_OPTIONS.get(method, None)

    for s_idx, start in enumerate(STARTS):
```

```
        bl = ExcelElements(EXCEL_PATH).create_beamline()[:A_BEAMLINE_LEN]
        opti = beamOptimizer(bl, PARTICLES.copy())

        traj = [start.copy()]
        def cb(xk, *args, **kwargs):
            traj.append(np.array(xk, dtype=float).copy())

        SP = {"I":  {"bounds": CURRENT_BOUNDS, "start": float(start[0])},
              "I2": {"bounds": CURRENT_BOUNDS, "start": float(start[1])}}

        res = opti.calc(method, A_VARS, SP, copy.deepcopy(A_OBJ),
                        jac=jac, options=opts, callback=cb)

        traj.append(np.array(res.x, dtype=float))
        method_trajs.append({"path": np.array(traj),
                             "start": start.copy(),
                             "final_mse": float(res.fun),
                             "nit": int(getattr(res, 'nit', -1))})
        print(f"{method:14s} start{s_idx+1} ({start[0]:.2f},{start[1]:.2f})"
              f"  steps={len(traj):3d}  MSE={res.fun:.2e}  x={np.round(res.
 ↪x,4)}")

    trajectories[method] = method_trajs

with open(os.path.join(out_dir, "scenario_A_trajectories.pkl"), "wb") as f:
    pickle.dump({"trajectories": trajectories, "starts": STARTS}, f)
print("\n  trajectories saved (5 methods * 5 starts)")
```

comparison with gradient descent and newton's method

[ ]:
```
# use gradient descent to optimize
# def grad_descent
```